PORTFOLIO

SLMAN AL-ODANI

NHL STENDEN EMMEN
Computer Science
Student Number : 5638364
Number of EC's : 240
Slman.al-odani@student.nhlstenden.com
2025-2029

Introduction:

As a Computer Science student, this portfolio reflects the journey I've taken to explore, build, and understand the digital world. From foundational programming concepts to hands-on projects, each section showcases not just what I've learned, but how I've applied it. Whether it's solving problems through code, analyzing systems, or experimenting with new technologies, I've aimed to approach each challenge with curiosity and persistence. This collection is a snapshot of my growth and a starting point for where I hope to go next.

My GitHub name is slmanalodani.

Table of Contents

Introduction: 1
Computer science assignments: 3
Assignments Week 3.1: 4
Assignments Week 3.2: 10
Assignments Week 3.3: 18
Assignments Week 3.5: 24
Assignments Week 3.6: 29
Assignments Week 3.7: 35
Solo Innovate: 41
Embedded Systems assignments: 41
Assignments Week 3.1: 42
Assignments Week 3.2: 44
Assignments Week 3.3: 48
Assignments Week 3.4: 53
Assignments Week 3.5: 57
Assignments Week 3.6: 63

Computer science assignments:

Assignments Week 3.1:

1. Smart Cafe Helper:

Smart Café Helper

Create a program that assists customers in a café.

Functionality:

1. **Order Handling**:
   Customers type in their orders, and the program responds appropriately regardless of how the order is entered. For example:
   - Input: COFFEE
   - Output: Okay, I will prepare coffee.
2. **Welcome Message**:
   When the program starts, it plays a customizable welcome message. For instance, the default message might be:
   - Welcome to Smart Café!
     The program should allow the user to transform this message by inserting ... between every character, producing:
   - W...e...l...c...o...m...e... t...o... S...m...a...r...t... C...a...f...e...!
3. **Emojis for Keywords**:
   The helper detects certain keywords in the customer's order and responds with corresponding emojis:
   - coffee → ☕
   - tea → 🍵
   - cake → 🍰
     Add two more examples of keywords and their corresponding emojis.
4. **Energy Calculation**:
   The program calculates energy using the formula ($E = mc^2$), based on the weight (in grams) of the food or drink. The customer inputs the weight, and the helper calculates the energy accordingly.
5. **Order Total with Tip**:
   The program calculates the total price of the customer's order, including a tip. The customer selects a tip percentage (e.g., 10%, 15%, 20%), and the program provides the final amount.

Requirements:

1. **Use of Functions**:
   Each feature must be implemented in a separate function for modularity.
2. **Loops and Conditionals**:
   Ensure the program can handle multiple orders in one session. Customers should be able to place orders repeatedly until they choose to exit.

The Code:

```
import sys

def main():
    welcome()
    total = 0
    while True:
        total = take_order(total)

def welcome():
    # welcome message
    print("Welcome to Smart Cafe!")
    welcome = input("Would you like to customize the welcome message?
(yes/no): ").lower()
    match welcome:
        case "yes":
            message = input("Enter your custom message: ")
            message = "...".join(ch for ch in message)
            print(f"Transformed welcome message: {message}")


def take_order(total):

    # order handeling
    order = input("Please enter you order or type \'done\' to finish:
").strip().lower()

    match order:
        case "coffee":
            print(f"Okay, I will prepare {order}", order.replace("coffee",
"☕"))
            calculate_energy()
            total += c_price()
            return total

        case "cake":
            print(f"Okay, I will prepare {order}", order.replace("cake",
"🍰"))
            calculate_energy()
            total += c_price()
            return total

        case "tea":
            print(f"Okay, I will prepare {order}", order.replace("tea",
"🍵"))
```

```
            calculate_energy()
            total += c_price()
            return total

        case "cookies":
            print(f"Okay, I will prepare {order}", order.replace("cookies",
"🍪"))
            calculate_energy()
            total += c_price()
            return total

        case "ice cream":
            print(f"Okay, I will prepare {order}", order.replace("ice cream",
"🍦"))
            calculate_energy()
            total += c_price()
            return total

        case "done":
                print(f"Your total is ${total: .2f}")
                tip(total)
                print("Thank you for visiting Smart Cafe!")
                sys.exit()
        case _:
            print("Sorry, we don't have that item.")
            return total

def calculate_energy():
    # calculate energy
    answer = input("Would you like to calculate energy ? (yes/no): ").lower()
    match answer:
        case "yes":
            weight = float(input("Enter the weight in grams: "))
            #convert from gram to kg
            weight = weight / 1000
            c = 300000000
            e = weight * (c * c)
            print(f"Energy: {e: .2e} Joules.") # .2e for the scientific
notation

def c_price():
    while True:
        price = input("Enter the price of this item: ").strip()
        price = price.replace("$", "")

        try:
```

```
            return float(price)
        except ValueError:
            print("Invalid price. Please enter a number like 3.50 or $4.")

def tip(total):

        percent = input("How much tip would you like to add? (10%, 15%, 20%):
").strip()
        #remove the % sign
        percent = percent.replace("%", "")
        #convert to percent
        try:
            percent = float(percent) / 100
            tip = total * percent
            total = total + tip
            print(f"with tip, your total is ${total: .2f}")
        except ValueError:
            print("Invalid input")
            sys.exit()
main()
```

2. Vanity Plates goes Flask:

   In this assignment, you need to implement a program that checks whether a vanity plate is valid based on a set of rules. The program should ask the user for a plate, validate it, and then output **“Valid”** or **“Invalid.”** The core logic must be written inside an `is_valid` function, which returns `True` or `False` depending on whether the plate meets all requirements.

   You then have to implement the same validation logic in a Flask application, where the user enters a plate through a web form and the server responds with whether the plate is valid.

   To be considered **valid**, a vanity plate must follow **all** of these rules:

   1. The plate must start with at least two letters

   - Examples: **CS50**, **AB123**, **HELLO**
   - Not allowed: **5CS50**, **A1234**

   2. Plates must be between 2 and 6 characters long

   - Minimum: **2 characters**

- Maximum: **6 characters**

3. Numbers cannot appear in the middle

- Once a number appears, **all remaining characters must be numbers**.
- Example of valid: **CS50**, **AB123**
- Example of invalid: **CS5A2**, **HEL0O**

4. The first number cannot be zero

- Valid: **CS50**, **AB123**
- Invalid: **CS01**, **AB012**

5. No punctuation or special characters

- Only **letters and numbers** allowed.
- Invalid: **CS!50**, **HELLO-1**, **A B123**

The code for the flask web app:

```
from flask import Flask, render_template, request
import sqlite3

app = Flask(__name__)

#----------- Database Setup-------------------------
def init_db():
    conn = sqlite3.connect("plates.db")
    cur = conn.cursor()
    cur.execute("CREATE TABLE IF NOT EXISTS plates (id INTEGER PRIMARY KEY,
text TEXT UNIQUE)")
    conn.commit()
    conn.close()

#------------Validation function--------------------

def is_valid(s):
    return (
        min_max(s)
        and first_two_letters(s)
        and no_leading_zero(s)
        and numbers_at_end(s)
    )

def min_max(s):
```

```
    return 2 <= len(s) <= 6

def no_leading_zero(s):
    for ch in s:
        if ch.isdigit():
            return ch != "0"
    return True

def first_two_letters(s):
    return s[0:2].isalpha()

def numbers_at_end(s):
    numbers_started = False
    for ch in s:
        if ch.isdigit():
            numbers_started = True
        else:
            if numbers_started:
                return False
    return True

# main route
@app.route("/", methods=["GET", "POST"])
def index():
    message = ""
    if request.method == "POST":
        plate = request.form.get("plate", "").strip()
        # validate the  plate
        if not is_valid(plate):
            message = "Invalid plate"
        else:
            try:
                #insert into the db
                conn = sqlite3.connect("plates.db")
                cur = conn.cursor()
                cur.execute("INSERT INTO plates (text) VALUES (?)", (plate,))
                conn.commit()
                conn.close()
                message = "Saved!"
            except:
                message = "Plate already exists"

    #fetch for plates
    conn = sqlite3.connect("plates.db")
    cur = conn.cursor()
    cur.execute("SELECT text FROM plates")
    rows = cur.fetchall()
```

```
    plates = [row[0] for row in rows]
    conn.close()
    return render_template("index.html", message=message, plates=plates)

if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

The html code:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Vanity Plates</title>
    </head>
    <body>
        <h1>Vanity Plates</h1>
        <form method="POST">
            <input name="plate" placeholder="Enter plate">
            <button type="submit">Save</button>
        </form>
        <p>{{ message }}</p>
        <h2>Saved Plates</h2>
        <ul>
            {% for p in plates %}
                <li>{{ p }}</li>
            {% endfor %}
        </ul>
    </body>
</html>
```

Assignments Week 3.2:

1. Survival Simulator:

You are going to create a program that is a survival simulator for the modern world. In this simulator, the user must solve various challenges with knowledge of file formats,
nutrition, mathematics, programming, and general logic. The simulator will be a combination of puzzles and interactive choices.
The user wakes up in an abandoned supermarket with no idea how they got there. To escape, they need to complete all kinds of tasks.

The code:

```
import random

def main():
    welcome()

    # Track completed puzzles
    completed = {
        1: False, 2: False, 3: False, 4: False,
        5: False, 6: False
    }

    # Main game loop
    while True:
        option = menu()
        option = selected_puzzle(option, completed)

        # If option is None, skip running a puzzle
        if option is not None:
            run_puzzle(option, completed)


def welcome():
    print("Welcome to the Supermarket Survivor!")
    print("You are in an abandoned supermarket and must solve puzzles to
escape!")
    print("")


def menu():
    BLUE = "\033[34m"
    RESET = "\033[0m"
```

```
    print("Choose an option:")
    print(f"{BLUE}1. {RESET} File Extensions: What's in this file?")
    print(f"{BLUE}2. {RESET} Math Interpreter: Unlock the safe")
    print(f"{BLUE}3. {RESET} Nutrition Facts: Prepare a meal")
    print(f"{BLUE}4. {RESET} CamelCase Decoder: Decode the message")
    print(f"{BLUE}5. {RESET} Emojize: Guess the clue?")
    print(f"{BLUE}6. {RESET} Guessing Game: Guess the secret code")
    print(f"{BLUE}7.{RESET} Exit")

    while True:
        try:
            option = int(input("Make a choice (1-7): "))
            if 1 <= option <= 7:
                return option
            print("Enter a valid number!")
        except ValueError:
            print("Enter a valid number!")


def selected_puzzle(option, completed):

    # Exit option
    if option == 7:
        if all(completed.values()):
            print("You solved ALL puzzles! You escape the supermarket!")
            exit()
        else:
            print("You cannot exit yet. Finish all puzzles first!")
            return None

    # Check if puzzle exists
    if option not in completed:
        print("Invalid puzzle.")
        return None

    # Prevent replaying puzzles
    if completed[option]:
        print("You already completed this puzzle. Choose another one.")
        return None

    return option


def run_puzzle(option, completed):
    if option == 1:
        file_extensions()
```

```
    elif option == 2:
        math_interpreter()
    elif option == 3:
        nutrition_facts()
    elif option == 4:
        camelCase_decoder()
    elif option == 5:
        emojize()
    elif option == 6:
        guessing_game()


    completed[option] = True
    print(f"Puzzle {option} completed!\n")


def file_extensions():
    print("You find a file named 'cat.jpg'.")
    while True:
        name = input("Type the name of the file to check its extension:
").lower()
        if name == "cat.jpg":
            print("This is a JPG file containing a cat image!")
            return
        else:
            print("Wrong file type!")


def math_interpreter():
    print("A safe unlocks only with the correct mathematical calculation.")

    while True:
        try:
            parts = input("Enter the mathematical expression (e.g, 2 + 2 * 2):
").split()

            if len(parts) == 3:
                a, b, c = parts
                a = float(a)
                c = float(c)

                if b == "+":
                    result = a + c
                elif b == "-":
                    result = a - c
                elif b == "*":
```

```
                result = a * c
            elif b == "/":
                if c == 0:
                    print("Cannot divide by zero.")
                    continue
                result = a / c
            else:
                print("Invalid operator.")
                continue

            print(f"The result is: {result}. The safe is open!")
            return

        elif len(parts) == 5:
            a, b, c, d, e = parts
            a = float(a)
            c = float(c)
            e = float(e)

            operators = ["+", "-", "*", "/"]

            if b not in operators or d not in operators:
                print("Invalid operator.")
                continue

            # First operation
            if b == "+":
                temp = a + c
            elif b == "-":
                temp = a - c
            elif b == "*":
                temp = a * c
            elif b == "/":
                if c == 0:
                    print("Cannot divide by zero.")
                    continue
                temp = a / c

            # Second operation
            if d == "+":
                result = temp + e
            elif d == "-":
                result = temp - e
            elif d == "*":
                result = temp * e
            elif d == "/":
```

```
                    if e == 0:
                        print("Cannot divide by zero.")
                        continue
                    result = temp / e

                print(f"The result is: {result}. The safe is open!")
                return

            else:
                print("Invalid expression format.")

        except ValueError:
            print("Invalid expression. Try again.")

def nutrition_facts():
    print("You need to prepare a meal with 500 calories to proceed.")
    print("Available options: apple, banana, cookie, sandwich, water")
    total = 0
    items = {
        "apple": 95, "banana": 105, "cookie": 150, "sandwich": 250, "water": 0
    }

    while True:
        try:
            item = input("Choose an item to add to your meal:
").strip().lower()
            if item not in items:
                print("Invalid option !")
                continue
            total += items[item]
            print(f"{item} added! Total calories: {total}")

            if total >= 500:
                print("Congratulations! Your meal is complete, and you have
enough energy")
                return
        except:
            pass

def camelCase_decoder():
    print("You find a message in CamelCase: 'EscapeNowThroughDoor'.")
    while True:
        message = input("Type the CamelCase message to decode it: ").strip()
        if message != "EscapeNowThroughDoor":
            print("Invalid message, try again!")
            continue
```

```
        decoded = ""

        for letter in message:
            if letter.isupper() and decoded != "":
                decoded += " " + letter
            else:
                decoded += letter
        print(f"Decoded message: {decoded}")
        return

def emojize():
    print("You find a clue in emojis: 🧀 🍇 🌽")
    foods = ["cheese", "grapes", "corn" ]
    while True:
        answer = input("What do these emojis mean? (Hint: food):
").strip().lower()
        # Check if ALL foods appear in the answer
        if all(item in answer for item in foods):
            print("Correct! You receive a key.")
            return
        else:
            print("That's not correct. Try again")

def guessing_game():
    print("A secret code (between 1 and 10) must be guessed.")

    while True:  # This loop allows replaying the whole game
        secret = random.randint(1, 10)
        attempts = 3

        while attempts > 0:
            try:
                answer = int(input(f"Guess the code (attempts left:
{attempts}): "))

                if answer > secret:
                    attempts -= 1
                    print(f"Too high! (attempts left: {attempts})")

                elif answer < secret:
                    attempts -= 1
                    print(f"Too low! (attempts left: {attempts})")

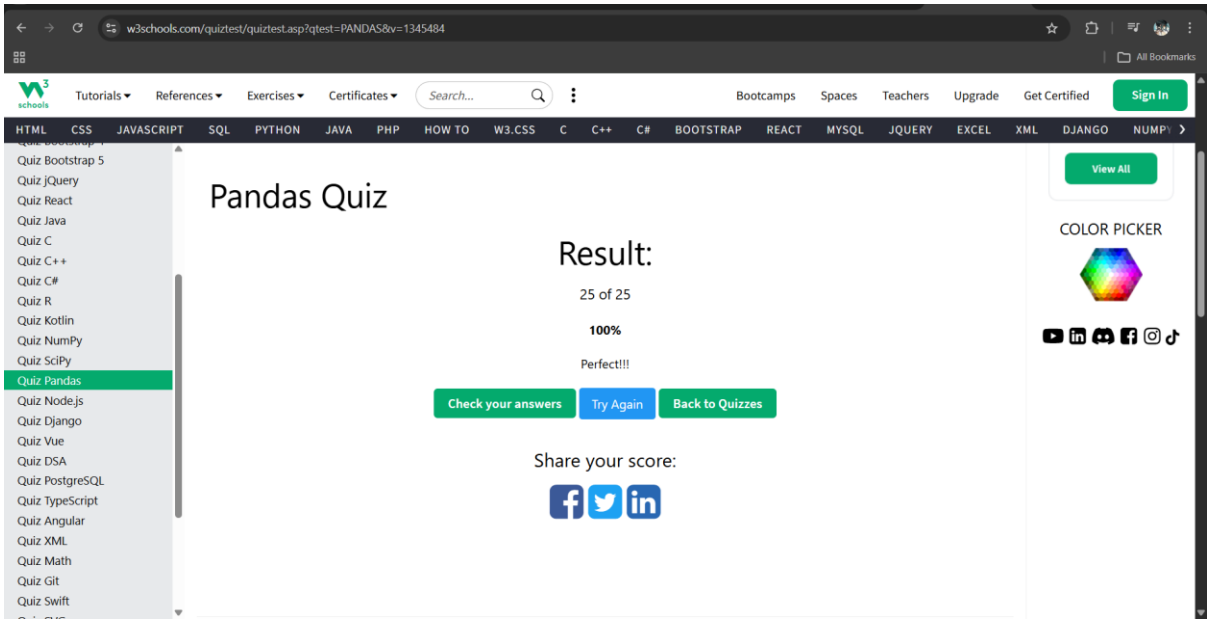
                else:
                    print("Correct! The door opens.")
                    return  # Puzzle completed
```

```
            except ValueError:
                print("NUMBERS PLEASE!")

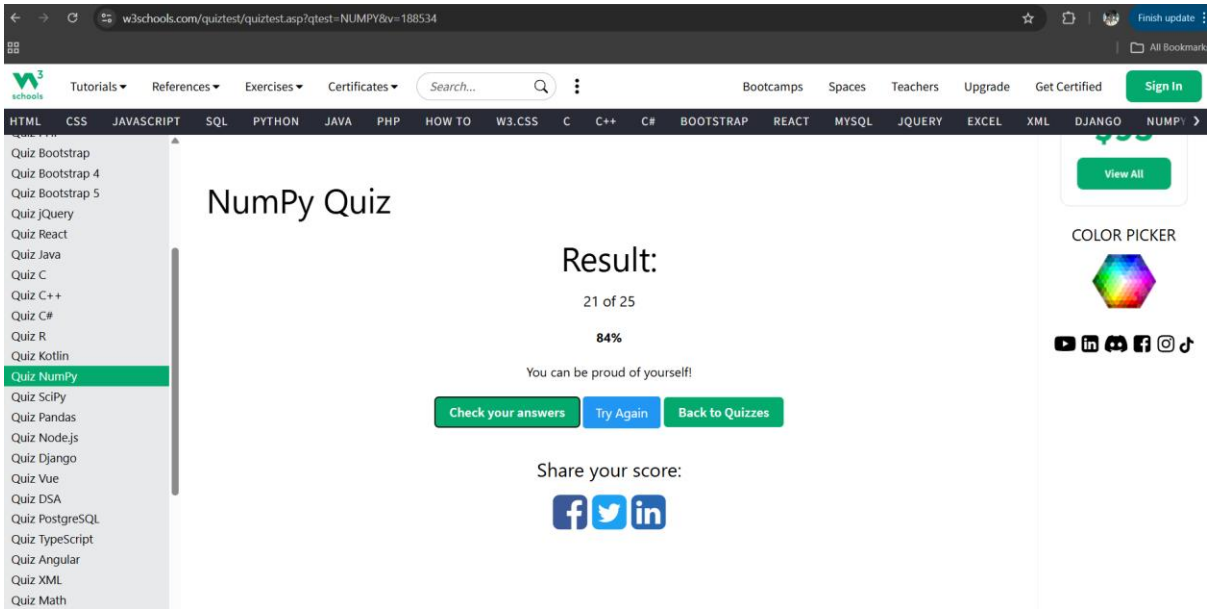
        # If the player used all attempts
        print("You ran out of attempts! The code resets... try again.\n")
        # Loop restarts: new secret number, new attempts

if __name__ == "__main__":
    main()
```

3. Pandas Score:



4. NumPy Score:



Assignments Week 3.3:

1. Testing my twttr:

```
def main():
    word = input("Input: ").strip()
    print("Output: ", end="")
    shorten(word)
    print()

def shorten(word):
    filtered = ""
    remove = [
        "a", "A",
        "i", "I",
        "o", "O",
        "e", "E",
        "u", "U"
    ]
    for ch in word:
        if ch not in remove:
            filtered += ch
            print(ch, end="")

    return filtered

if __name__ == "__main__":
    main()
```

```
from twttr import shorten

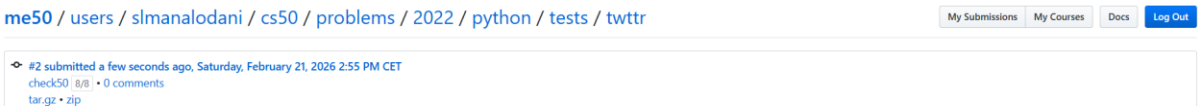
def test_remove_a():
    assert shorten("sal") == "sl"

def test_capitalized():
    assert shorten("SAL") == "SL"
```

```
def test_puncuation():
    assert shorten("!@#$") == "!@#$"

def test_numbers():
    assert shorten("123") == "123"
```

https://submit.cs50.io/check50/fab7907553d832aa2861efdd8456488230cd4277



2. Back to the bank:

In a file called bank.py, reimplement Home Federal Savings Bank from Problem Set 1. Then, in a file called test_bank.py, implement three or more functions that collectively test your functions.

The code for bank.py:

```
def main():
    greeting = input("Greeting: ").strip().lower()
    cost = value(greeting)
    print(f"${cost}")

def value(greeting):
    match greeting:
        case _ if greeting.startswith("hello"):
            return 0
        case _ if greeting.startswith("h"):
            return 20
        case _:
            return 100

if __name__ == "__main__":
    main()
```

The code for test_bank,py:

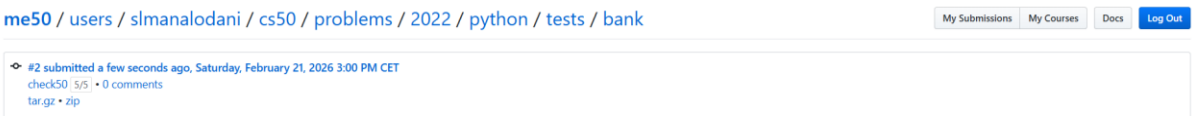
```
from bank import value
```

```python
def test_hello():
    assert value("hello") == 0

def test_start_h():
    assert value("hsSdsa") == 20

def test_other_letters():
    assert value("sadas") == 100

def test_case_insensitive():
    assert value("HELLO") == 0
```



https://submit.cs50.io/check50/fc1b8fca3984d380bd40d95a171a7016b49af01a

3. Re-questing a vanity plate:

In a file called plates.py, reimplement Vanity Plates from Problem Set 2.

Then, in a file called test_plates.py, implement four or more functions that collectively test your functions.

The plates.py code:

```python
def main():
    plate = input("Plate: ")
    if is_valid(plate):
        print("Valid")
    else:
        print("Invalid")

def is_valid(s):
     return (
          min_max(s)
          and no_leading_zero(s)
          and first_two_letters(s)
          and numbers_at_end(s)
          )
```

```
def min_max(s):
      # max 6 and min 2
      if 2 <= len(s) <= 6:
           return True
      else:
           return False

def no_leading_zero(s):
     for ch in s:
          if ch.isdigit():
               if ch != "0":
                   return True
               else:
                    return False
     return True

def first_two_letters(s):
     if s[0:2].isalpha():
          return True
     else:
          return False

def numbers_at_end(s):
     numbers_started = False
     for ch in s:
          if ch.isdigit():
               numbers_started = True
          else:
               if numbers_started:
                    return False
     return True

if __name__ == "__main__":
     main()
```

The code for test_plates.py:

```
from plates import is_valid

def test_min_and_max():
    assert is_valid("A") == False
    assert is_valid("ASDA12312") == False
```

```python
def test_no_leading_zero():
    assert is_valid("ASD01") == False

def test_first_must_two_letters():
    assert is_valid("12ADSA") == False

def test_number_at_end():
    assert is_valid("ASA23A") == False

def test_alphabetic():
    assert is_valid("AS@#f") == False
    assert is_valid("A12345") == False
```



https://submit.cs50.io/check50/c8b811f0f4a2c05af43c17f3bc9ddc351bc20d1d

4. Nutrition Facts:

In a file called nutrition.py, implement a program that prompts users to input a fruit (case-insensitively) and then outputs the number of calories in one portion of that fruit, per the FDA's poster for fruits.

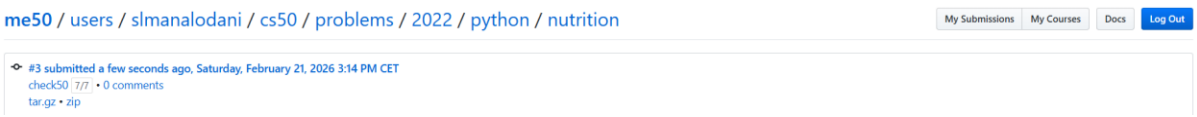
The Code:

```python
def main():

    # dic for the fruit with thier associated calories
    calories = {
        "apple": 130, "avocado": 50, "banana": 110,
        "cantaloupe": 50, "grapefruit": 60, "grapes": 90,
        "honeydew melon": 50, "kiwifruit": 90, "lemon": 50,
        "lime": 20, "nectarine": 60, "orange": 80,
        "peach": 60, "pear": 100, "pineapple": 50,
        "plums": 70, "strawberries": 50, "sweet cherries": 100,
        "tangerine": 50, "watermelon": 80
    }
    # ask the user for an item
```

```
    while True:
        try:
            item = input("Item: ").strip().lower()
            if item in calories:
                return print("Calories:", calories[item])
            else:
                return
        except:
            pass

main()
```



Assignments Week 3.5:

1) Scavenger Hunt

Assignment 1: Validate Email Address

Write a function is_valid_email(email) that checks whether an email address is valid. An email is valid if:

Requirements:

- It contains exactly one @ symbol.
- There is at least one character before and after the @.
- There is at least one . in the domain after the @.
- You have to return a True or False

Assignment 1: write your function here:

My Code:

```
def is_valid_email(email):
        if email.count("@") !=1:
            return False
        local, domain = email.split("@")
        if len(local) == 0:
```

```
            return False
        if len(domain) == 0:
            return False
        if "." not in domain:
            return False
        return True
```

Assignment 2: FizzBuzz

Write a function fizzbuzz(n) that:

Returns "Fizz" if n is divisible by 3. Returns "Buzz" if n is divisible by 5. Returns "FizzBuzz" if n is divisible by both 3 and 5. Otherwise, returns n as a string. def fizzbuzz(n): if n / 3 == 1: return "Fizz" elif (n / 5) == 1: return "Buzz" elif (n / 3 == 1) and (n / 5 == 1): return "FizzBuzz" else: return n

My Code:

```
def fizzbuzz(n):
    if n % 3 == 0 and n % 5 == 0:
        return "FizzBuzz"
    elif n % 3 ==0:
        return "Fizz"
    elif n % 5 == 0:
        return "Buzz"
    else:
        return str(n)
```

Assignment 3: Palindrome Checker

Write a function is_palindrome(s) that checks whether a word or sentence is a palindrome. A palindrome reads the same forward and backward, ignoring capitalization and spaces.

My code:

```
def is_palindrome(s):
    s = s.lower().replace(" ","")
    if s == s[::-1]:
        return True
    else:
        return False
```

Assignment 4: Find the longest word

Write a function longest_word(sentence) that returns the longest word in a sentence. If multiple words have the same length, return the first one. def longest_word(sentence): counter = 0 w = "" s = sentence.split(" ") for word in s: if len(word) > len(w): counter = len(word) return word

My code:

```
def longest_word(sentence):
    highest_length = 0
    biggest = ""
    sentence = sentence.split()
    for word in sentence:
        if len(word) >= highest_length:
            highest_length = len(word)
            biggest = word
    return biggest
```

Assignment 5: Unit Testing a calculator

Write a simple unittest for the calculator with the following methods: def add(a, b): return a + b

def subtract(a, b): return a - b

def multiply(a, b): return a * b

def divide(a, b): if b == 0: return "Error: Division by zero" return a / b try: assert add(1, 2)

My code:

```
try:
    assert add(1, 2) == 3
    assert subtract(2, 2) == 0
    assert multiply (2, 2) == 4
    assert divide(4, 2) == 2
    print("All tests passed!")
except AssertionError as e:
    print(e)
```

2) CS50 P-Shirt

In a file called shirt.py, implement a program that expects exactly two command-line arguments:

- in sys.argv[1], the name (or path) of a JPEG or PNG to read (i.e., open) as input
- in sys.argv[2], the name (or path) of a JPEG or PNG to write (i.e., save) as output

The program should then overlay shirt.png (which has a transparent background) on the input after resizing and cropping the input to be the same size, saving the result as its output.

The code:

```
import sys
import os
from PIL import Image, ImageOps

def main():
    valid = ["jpg","jpeg", "png"]
    # check argument count
    if len(sys.argv) > 3:
        sys.exit("Too Many Command line Arguments")
    elif len(sys.argv) < 3:
        sys.exit("Too few Command Arguments")

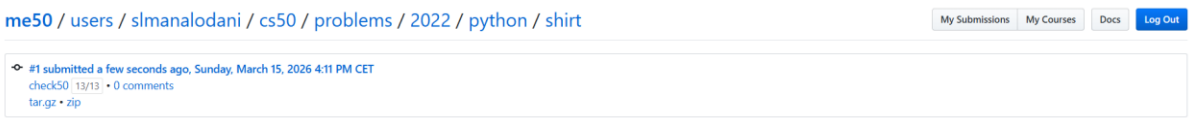
    first, extension_1 = sys.argv[1].lower().rsplit(".", 1)

    second, extension_2 = sys.argv[2].lower().rsplit(".", 1)

    if extension_1 not in valid or extension_2 not in valid:
        sys.exit("Not Valid Extensions")
    if extension_1 != extension_2:
        sys.exit("Use The Same Extensions!")
    # check whether the file exists
    if not os.path.isfile(sys.argv[1]):
        sys.exit("Input does not exist")
    # open the shirt
    shirt = Image.open("shirt.png")
    # open the photo
    photo = Image.open(sys.argv[1])
    # resize/crop the photo so it matches the shirt's size exactly
    photo = ImageOps.fit(photo, shirt.size)
    # paste the shirt on top of the resized photo
    # using the shirt itself as the mask preserves transparency
    photo.paste(shirt,shirt)
```

```
    photo.save(sys.argv[2])

if __name__ == "__main__":
    main()
```



https://submit.cs50.io/check50/376f6dcf147cad70d8e201b5b987811a53c59f11

Assignments Week 3.6:

1) Tic-Tac-Toe

   In this assignment, I implemented complete logic and artificial intelligence for a Tic-Tac-Toe game. I built functions to manage the game state, determine valid moves, detect winners, and evaluate terminal positions. The core of the project was implementing the Minimax algorithm, allowing the AI to simulate all possible future game states and always choose the optimal move. This ensures the computer plays perfectly and can never be beaten. The project strengthened my understanding of search algorithms, recursion, and decision-making in AI.

   The code:

```
"""
Tic Tac Toe Player
"""

import math

X = "X"
O = "O"
EMPTY = None


def initial_state():
    """
    Returns starting state of the board.
    """
    return [[EMPTY, EMPTY, EMPTY],
            [EMPTY, EMPTY, EMPTY],
            [EMPTY, EMPTY, EMPTY]]
```

```python
def player(board):
    """
    Returns player who has the next turn on a board.
    """
    count_x = 0
    count_o = 0
    # loop through every cell
    for row in board:
        for cell in row:
            if cell == "X":
                count_x += 1
            elif cell == "O":
                count_o += 1
    if count_x > count_o:
        return "O"
    elif count_x == count_o:
        return "X"
    # if somehow the board is invalid return x as safety net
    else:
        return "X"


def actions(board):
    """
    Returns set of all possible actions (i, j) available on the board.
    """
    # empty set
    actions = set()
    # loop to get the coordnates for the empty cells
    for i in range(len(board)):
        for j in range(len(board[i])):
            if board[i][j] == None:
                actions.add((i, j))
    return actions



def result(board, action):
    """
    Returns the board that results from making move (i, j) on the board.
    """
    # validate the action
    valid_actions = actions(board)

    if action not in valid_actions:
        raise Exception("Invalid action")
```

```
    # deep copy of the board
    new_board = [row.copy() for row in board]

    # determine the current player
    current_player = player(board)

    # apply the sympol
    i, j = action
    new_board[i][j] = current_player
    # return the new board
    return new_board

def winner(board):
    """
    Returns the winner of the game, if there is one.
    """
    # three conditions horizonal vertical diagonal
    # check horizontal
    for row in board:
        if row[0] is not None and row[0] == row[1] == row[2]:
            return row[0]
    # check vertical
    for col in range(3):
        if board[0][col] is not None and board[0][col] == board[1][col] ==
board[2][col]:
            return board[0][col]
    # check diagonal top left to bottom right
    if board[0][0] is not None and board[0][0] == board[1][1] == board[2][2]:
        return board[0][0]
    # check diagonal top right to bottom left
    if board[0][2] is not None and board[0][2] == board[1][1] == board[2][0]:
        return board[0][2]
    return None


def terminal(board):
    """
    Returns True if game is over, False otherwise.
    """
    # if the game is over bc someone won
    if winner(board) != None:
        return True
    # check if all cells are filled
    counter = 0
    for row in board:
```

```
        for cell in row:
            if cell != None:
                counter += 1
    if counter == 9:
        return True

    return False



def utility(board):
    """
    Returns 1 if X has won the game, -1 if O has won, 0 otherwise.
    """
    if terminal(board):
        # if x won
        if winner(board) == "X":
            return 1
        elif winner(board) == "O":
            return -1
        else:
            return 0



def minimax(board):
    """
    Returns the optimal action for the current player on the board.
    Uses the minimax algorithm to choose the best move.
    """

    # If the game is already over, there is no move to make
    if terminal(board):
        return None

    # Determine whose turn it is
    current = player(board)

    # If it's X's turn, we want to MAXIMIZE the score
    if current == "X":

        # best_score starts very low because X wants the highest score
        best_score = -math.inf
        best_move = None
```

```
        # Loop through all possible actions
        for action in actions(board):

            # Get the board that results from making this move
            new_board = result(board, action)

            # Recursively evaluate the score of this move
            score = min_value(new_board)

            # If this move is better than what we have, update best_move
            if score > best_score:
                best_score = score
                best_move = action

        return best_move

    # If it's O's turn, we want to MINIMIZE the score
    else:

        # best_score starts very high because O wants the lowest score
        best_score = math.inf
        best_move = None

        # Loop through all possible actions
        for action in actions(board):

            # Get the board that results from making this move
            new_board = result(board, action)

            # Recursively evaluate the score of this move
            score = max_value(new_board)

            # If this move is better (lower) than what we have, update
best_move
            if score < best_score:
                best_score = score
                best_move = action

        return best_move




def max_value(board):
    """
    Returns the maximum utility value for X.
```

```
    """

    # If the game is over, return the utility score
    if terminal(board):
        return utility(board)

    # Start with the lowest possible value
    v = -math.inf

    # Try every possible move
    for action in actions(board):

        # X wants the highest score, so take the max
        v = max(v, min_value(result(board, action)))

    return v



def min_value(board):
    """
    Returns the minimum utility value for O.
    """

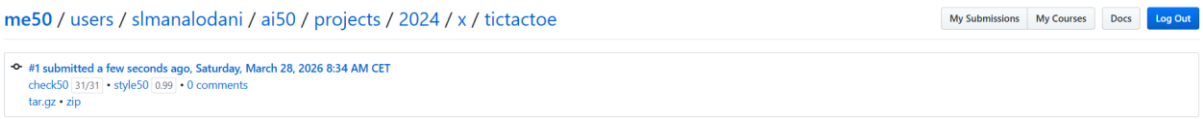
    # If the game is over, return the utility score
    if terminal(board):
        return utility(board)

    # Start with the highest possible value
    v = math.inf

    # Try every possible move
    for action in actions(board):

        # O wants the lowest score, so take the min
        v = min(v, max_value(result(board, action)))

    return v
```



https://submit.cs50.io/check50/fd22b5671162c3672cb466dcb91a38faa4112d1b

Assignments Week 3.7:

1) The Misleading Minesweeper

In this version of Minesweeper, exactly one revealed number is incorrect.
One cell, when clicked, will show a number that is one higher than the actual number of mines around it. But everything else in the game works exactly as expected.
Your AI will now have to learn how to detect inconsistencies in logic, identify the lying cell, and adapt its strategy to still try and win the game.

1. Modify the game logic so that one safe cell (not a mine) gives an incorrect mine count when revealed (actual + 1).
2. Do not reveal to the player or AI which cell is lying.
3. Update your AI to:
   - Notice when the knowledge base becomes logically inconsistent (e.g., two overlapping sentences with contradictory conclusions).
   - Isolate the suspicious cell causing the contradiction.
   - Avoid using that cell in future logical deductions.
4. The AI should continue playing the game using adjusted reasoning and still try to win, despite the misleading information.

The Code:

```
import random

class Minesweeper():
    """
    Basic Minesweeper game.
    In this version, ONE safe cell lies by reporting +1 mines.
    """

    def __init__(self, height=8, width=8, mines=8):

        self.height = height
        self.width = width
        self.mines = set()
```

```
        # Create empty board
        self.board = []
        for i in range(self.height):
            row = []
            for j in range(self.width):
                row.append(False)
            self.board.append(row)

        # Randomly place mines
        while len(self.mines) != mines:
            i = random.randrange(height)
            j = random.randrange(width)
            if not self.board[i][j]:
                self.mines.add((i, j))
                self.board[i][j] = True

        # Track mines the player found
        self.mines_found = set()

        # Pick ONE safe cell that will lie
        safe_cells = [
            (i, j)
            for i in range(self.height)
            for j in range(self.width)
            if not self.board[i][j]
        ]
        self.liar_cell = random.choice(safe_cells)

    def nearby_mines(self, cell):
        """
        Returns number of mines around a cell.
        ONE special cell lies and returns +1.
        """

        (r, c) = cell
        count = 0

        # Check all neighbors
        for i in range(r - 1, r + 2):
            for j in range(c - 1, c + 2):

                if (i, j) == cell:
                    continue

                if 0 <= i < self.height and 0 <= j < self.width:
                    if self.board[i][j]:
```

```
                        count += 1

        # Lying cell returns wrong number
        if cell == self.liar_cell:
            return count + 1

        return count
class Sentence():
    """
    A logical statement like:
    {(1,2), (1,3), (2,2)} = 1
    meaning exactly 1 of these cells is a mine.
    """

    def __init__(self, cells, count):
        self.cells = set(cells)
        self.count = count

    def known_mines(self):
        # If all cells must be mines
        if len(self.cells) == self.count:
            return set(self.cells)
        return set()

    def known_safes(self):
        # If count is zero, all cells are safe
        if self.count == 0:
            return set(self.cells)
        return set()

    def mark_mine(self, cell):
        # Remove cell and reduce count
        if cell in self.cells:
            self.cells.remove(cell)
            self.count -= 1

    def mark_safe(self, cell):
        # Remove safe cell
        if cell in self.cells:
            self.cells.remove(cell)
class MinesweeperAI():

    def __init__(self, height=8, width=8):

        self.height = height
        self.width = width
```

```
        self.moves_made = set()
        self.mines = set()
        self.safes = set()

        # Sentences we know
        self.knowledge = []

        # Cells we don't trust (possible liar)
        self.suspicious = set()

    def mark_mine(self, cell):
        self.mines.add(cell)
        for s in self.knowledge:
            s.mark_mine(cell)

    def mark_safe(self, cell):
        self.safes.add(cell)
        for s in self.knowledge:
            s.mark_safe(cell)

    def add_knowledge(self, cell, count):
        """
        Main logic:
        - mark cell safe
        - build sentence from neighbors
        - detect contradictions
        - mark suspicious cells
        """

        self.moves_made.add(cell)
        self.mark_safe(cell)

        (r, c) = cell
        neighbors = set()

        # Collect neighbors
        for i in range(r - 1, r + 2):
            for j in range(c - 1, c + 2):

                if (i, j) == cell:
                    continue

                if 0 <= i < self.height and 0 <= j < self.width:
```

```
                    # If we already know it's a mine
                    if (i, j) in self.mines:
                        count -= 1

                    # Only add unknown + not suspicious
                    elif (i, j) not in self.safes and (i, j) not in
self.suspicious:
                        neighbors.add((i, j))

        # Add new sentence
        if len(neighbors) > 0:
            self.knowledge.append(Sentence(neighbors, count))

        # Loop until no new info
        changed = True
        while changed:
            changed = False

            new_safes = set()
            new_mines = set()

            # Check each sentence
            for s in self.knowledge:
                new_safes |= s.known_safes()
                new_mines |= s.known_mines()

            # Mark safes
            for s in new_safes:
                if s not in self.safes:
                    self.mark_safe(s)
                    changed = True

            # Mark mines
            for m in new_mines:
                if m not in self.mines:
                    self.mark_mine(m)
                    changed = True

            # Try to infer new sentences
            inferred = []
            for s1 in self.knowledge:
                for s2 in self.knowledge:
                    if s1 == s2:
                        continue

                    if s1.cells.issubset(s2.cells):
```

```
                    diff_cells = s2.cells - s1.cells
                    diff_count = s2.count - s1.count

                    # CONTRADICTION CHECK
                    if diff_count < 0 or diff_count > len(diff_cells):
                        # Mark all involved cells as suspicious
                        self.suspicious |= (s1.cells | s2.cells)
                        continue

                    inferred.append(Sentence(diff_cells, diff_count))

        # Add inferred sentences
        for s in inferred:
            if len(s.cells) > 0 and s not in self.knowledge:
                self.knowledge.append(s)

    def make_safe_move(self):
        """
        Pick a safe cell that:
        - we haven't clicked
        - is not suspicious
        """
        for cell in self.safes:
            if cell not in self.moves_made and cell not in self.suspicious:
                return cell
        return None

    def make_random_move(self):
        """
        Pick a random unknown cell that is not:
        - a mine
        - suspicious
        - already clicked
        """
        choices = []
        for i in range(self.height):
            for j in range(self.width):
                cell = (i, j)
                if (
                    cell not in self.moves_made
                    and cell not in self.mines
                    and cell not in self.suspicious
                ):
                    choices.append(cell)

        if len(choices) == 0:
```

```
            return None

        return random.choice(choices)
```

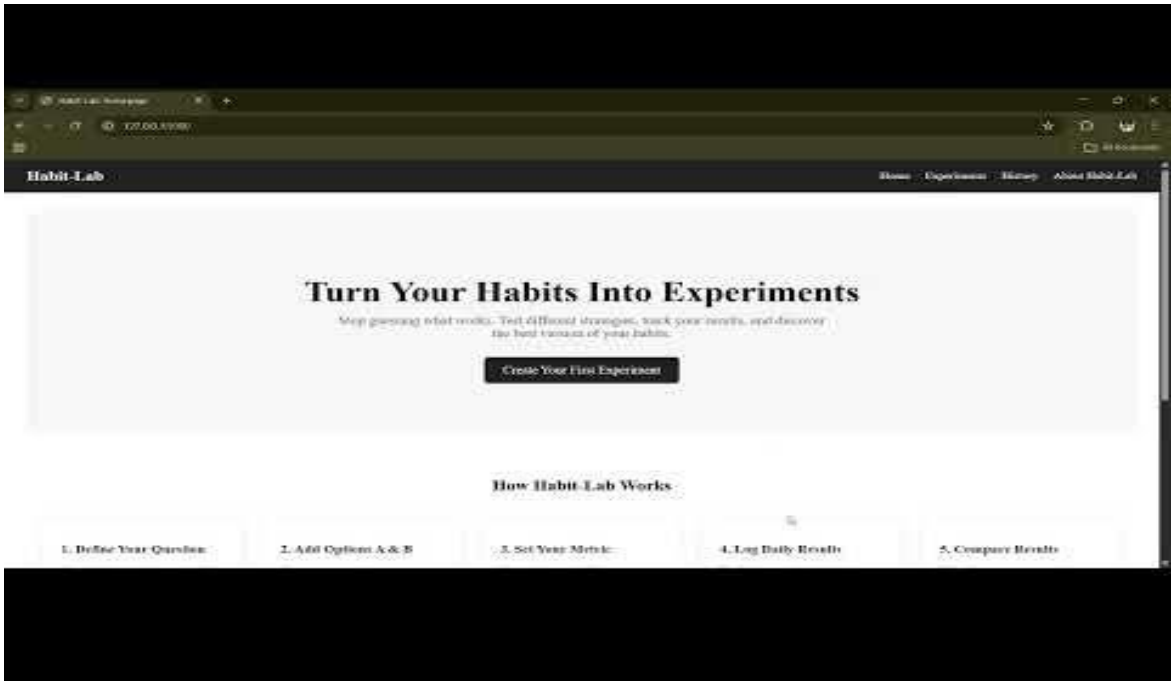
Solo Innovate:

Link to the git-hub:

https://github.com/slmanalodani/solo_innovate

Youtube link about the project:

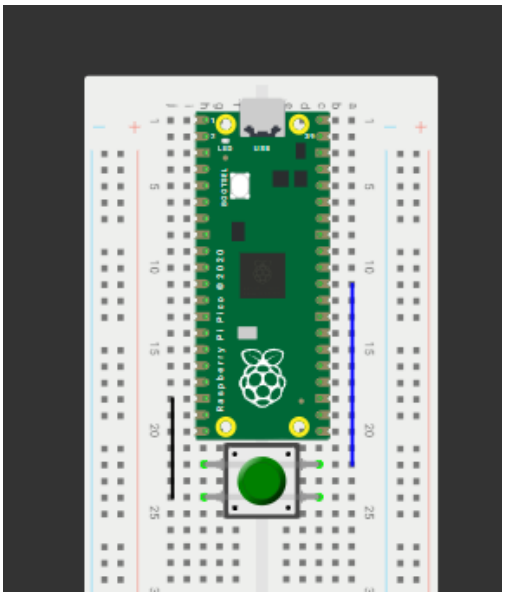
https://youtu.be/OOwzwSgEV4k



Embedded Systems assignments:

Assignments Week 3.1:

1. Add a hardware reset button to your Pico.



- Connect the RUN pin with the external button.

2. Blink:
   Create a program that makes the internal LED blink.
   The Code:

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)

while True:
    led.on()
    time.sleep(0.5)
    led.off()
    time.sleep(0.5)
```

3. Mario Pico:
   Create a program that uses Pico to make Mario pyramids.

   The code:

```
def main():
```

```
    # ask the user for height and keep asking for a number between 1 and 8
    while True:
        try:
            n = int(input("Height: "))
            if 1 <= n <= 8:
                break
        except:
            pass #ignore invalid input

        for i in range(n):
            numberspaces = n - 1 - i
            numberbricks = i + 1
            print_row(numberspaces, numberbricks)

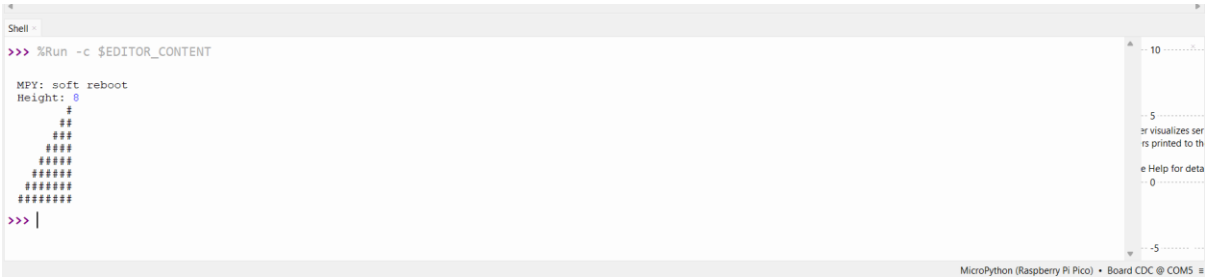
def print_row(spaces, bricks):
    # print spaces
    for _ in range(spaces):
        print(" ", end="")  # to avoid new lines with end=""

    # print bricks
    for _ in range(bricks):
        print("#", end="")

    print()  # after row printed move to next line


main()
```

Image from the terminal:



Assignments Week 3.2:

1. 7-Segments Voltmeter (0...10v):

The assignment is to create a voltmeter using the Pico and 7 segments 4-digit display.

The tools:

- Resistors
- Transistors
- 4-digits 7 segments display
- Breadboard
- Jumper wires

The code:

```
#           VoltMeter
# This is a voltmeter Project
# the program display the measured voltage
# in a 4 digit 7-segment display
# Author : Slman
# file   : voltMeter.py
# Date   : 18 / 02 / 2026
#-----------------------------------------
from machine import Pin, ADC
import utime

# volt divider resistors
R1 = 47000
R2 = 22000

#----SEGMENT PINS----
segA = Pin(21, Pin.OUT)
segB = Pin(20, Pin.OUT)
segC = Pin(19, Pin.OUT)
```

```
segD = Pin(18, Pin.OUT)
segE = Pin(17, Pin.OUT)
segF = Pin(16, Pin.OUT)
segG = Pin(14, Pin.OUT)
segDP = Pin(15, Pin.OUT)

segments = [segA, segB, segC, segD, segE, segF, segG, segDP]

#----DIGIT PINS----
digit2 = Pin(10, Pin.OUT)
digit3 = Pin(11, Pin.OUT)
digit4 = Pin(12, Pin.OUT)

digits = [digit2, digit3, digit4]

#----ADC SETUP----
adc = ADC(0) #GP26

#----SEGMENT PATTERNS FOR 0-9 + L ----
numbers = {
    0: [1,1,1,1,1,1,0,0],
    1: [0,1,1,0,0,0,0,0],
    2: [1,1,0,1,1,0,1,0],
    3: [1,1,1,1,0,0,1,0],
    4: [0,1,1,0,0,1,1,0],
    5: [1,0,1,1,0,1,1,0],
    6: [1,0,1,1,1,1,1,0],
    7: [1,1,1,0,0,0,0,0],
    8: [1,1,1,1,1,1,1,0],
    9: [1,1,1,1,0,1,1,0],
    "L":[0,0,0,1,1,1,0,0] # segments D, E, F
}

# ---- MAIN LOOP ----
def main():
    while True:
        v = read_voltage()
        display_number(v)

#----Function to display one digit----
def show_digit(value, digit_pin, show_dp=False):

    # turn all digits off
    for d in digits:
        d.value(0)
```

```
    # turn all segments off
    for seg in segments:
        seg.value(0)

    #load the pattern
    pattern = numbers[value]

    # Set segment
    for i in range(7): #A-G
        segments[i].value(pattern[i])

    # Decimal Point
    if show_dp:
        segDP.value(1)
    else:
        segDP.value(0)

    digit_pin.value(1) # turn this one on
    utime.sleep_ms(1)  # small delay for multiplexing

# ---- FUNCTION TO DISPLAY A FULL 3-DIGIT NUMBER ----
def display_number(value):

    # OVER LIMIT CHECK
    if value > 9.99:
        # show O on digit 2
        show_digit(0, digit2, show_dp=False)

        # show L on digit 3
        show_digit("L", digit3, show_dp=False)

        # Turn off digit 4
        digit4.value(0)
        for seg in segments:
            seg.value(0)

        return

    #----Normal number display----
    # convert number to string with 2 decimals
    s = "{:.2f}".format(value)

    # extract digits
    d2 = int(s[0])
    d3 = int(s[2])
```

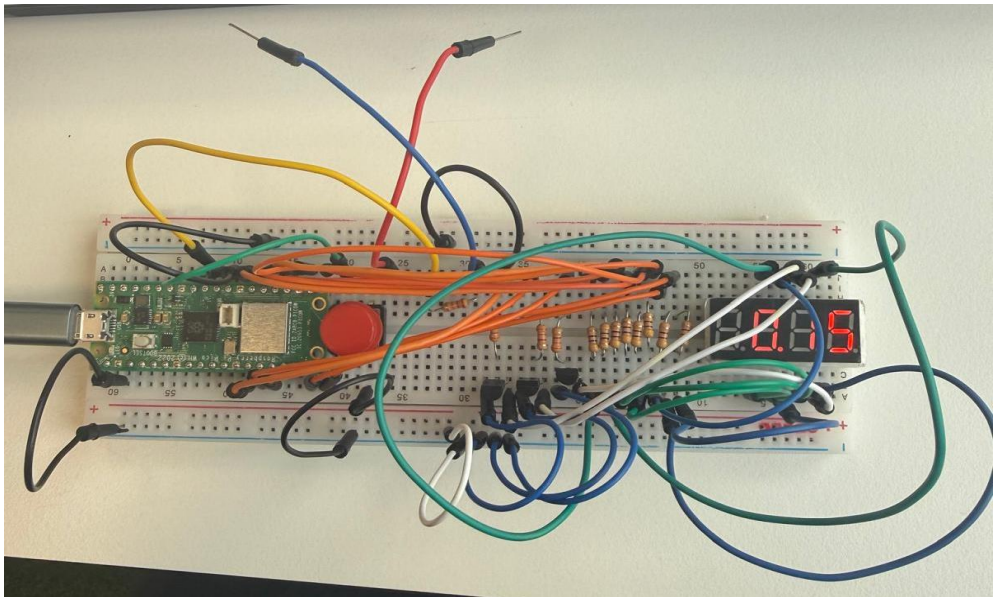
```
    d4 = int(s[3])

    # show digit
    show_digit(d2, digit2, show_dp=True) # Dp between digit 2 and 3
    show_digit(d3, digit3, show_dp=False)
    show_digit(d4, digit4, show_dp=False)

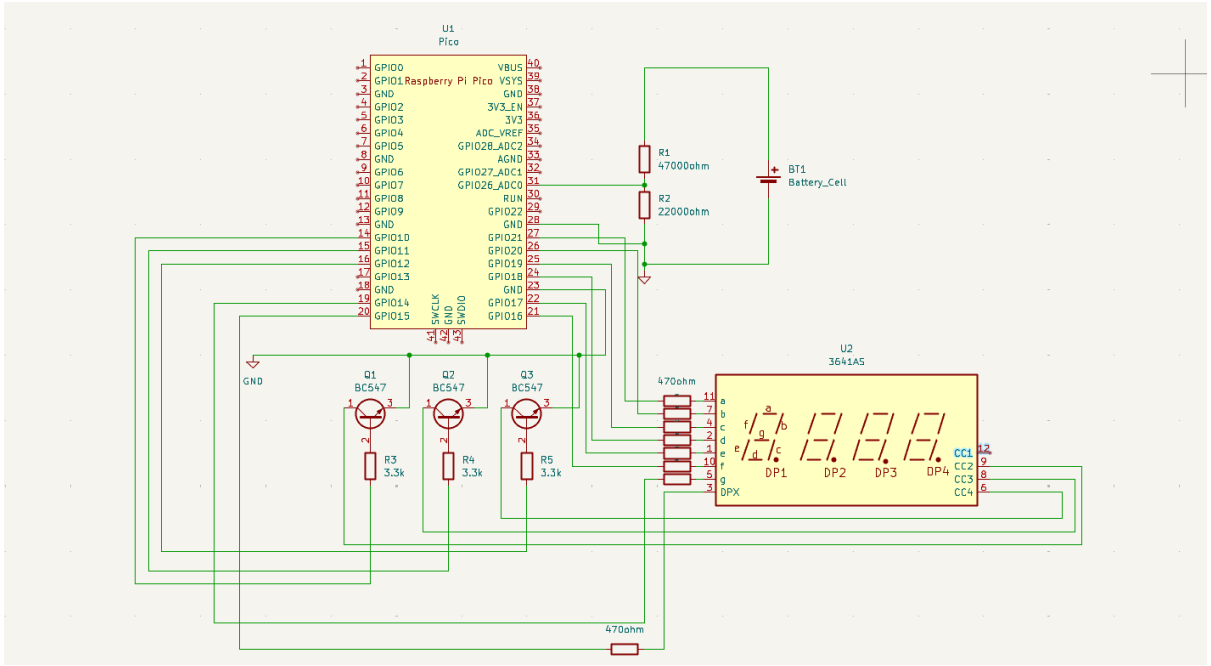
# ---- READ ADC AND CONVERT TO VOLTAGE ----
def read_voltage():
    # avg to reduce the noise
    samples = 100
    total = 0
    for _ in range(samples):
        total += adc.read_u16()
    raw = total / samples
    print(raw)
    voltage = (raw / 65535) * 3.3 # convert to 0–3.3V
    v_in = voltage / (R2/(R2 + R1))
    return v_in

main()
```

Image of the voltmeter:



Schematic: using KiCad



Assignments Week 3.3:

1.RTC and temperature on LCD-display:

The assignment is to show the time, date, and temperature on the LDC display.

The tools:

- Jumper wires
- RTC
- **16x2 I2C** LCD display
- LM 35 temperature sensor

The Code:

```
#-------------------------------------------------------
#    REAL TIME CLOCK AND TEMP
#    ======================
#
# This program uses the DS1307 RTC module to set and display
# the date, time and temp on LCD
#
# Author: Slman
# File  : temp_clock.py
# Date  : feb 2026
#-------------------------------------------------------
from machine import I2C, Pin, ADC
from lcd_api import LcdApi
from i2c_lcd import I2cLcd
import utime
import ds1302

I2C_ADDR = 0x27                 # I2C LCD address
NROWS = 2                       # Number of rows
NCOLUMNS = 16                   # Number of columns

i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
lcd = I2cLcd(i2c, I2C_ADDR, NROWS, NCOLUMNS)

ds = ds1302.DS1302(Pin(16), Pin(17), Pin(18))

# LM35 on ADC0 (GP26)
lm35 = ADC(Pin(26))

sel = input("Do you wish to set Date/Time ?: ")
if sel.upper() == "Y":
  print("Date time format is e.g. 2023, 9, 14, 4, 20, 22, 1, 0")
```

```
  Y = int(input("Enter Year    : "))
  M = int(input("Enter Month   : "))
  D = int(input("Enter Day     : "))
  day = int(input("Enter day no  : "))
  h = int(input("Enter Hour    : "))
  m = int(input("Enter Minutes : "))
  s = int(input("Enter Seconds : "))
  ds.date_time([Y,M,D,day,h,m,s])

# screen toggle
show_time = True
last_switch = utime.ticks_ms()

while True:

    # switch every 3 seconds
    if utime.ticks_diff(utime.ticks_ms(), last_switch) > 3000:
        show_time = not show_time
        lcd.clear()
        last_switch = utime.ticks_ms()

    if show_time:
        # --- TIME & DATE SCREEN ---
        (Y,M,D,day,hr,m,s)=ds.date_time()

        if s < 10: s = "0" + str(s)
        if m < 10: m = "0" + str(m)
        if hr < 10: hr = "0" + str(hr)
        if D < 10: D = "0" + str(D)
        if M < 10: M = "0" + str(M)

        lcd.move_to(0,0)
        lcd.putstr("Time:")
        lcd.move_to(6,0)
        lcd.putstr(str(hr) + ":" + str(m) + ":" + str(s))

        lcd.move_to(0,1)
        lcd.putstr("Date:")
        lcd.move_to(6,1)
        lcd.putstr(str(D) + "/" + str(M) + "/" + str(Y))

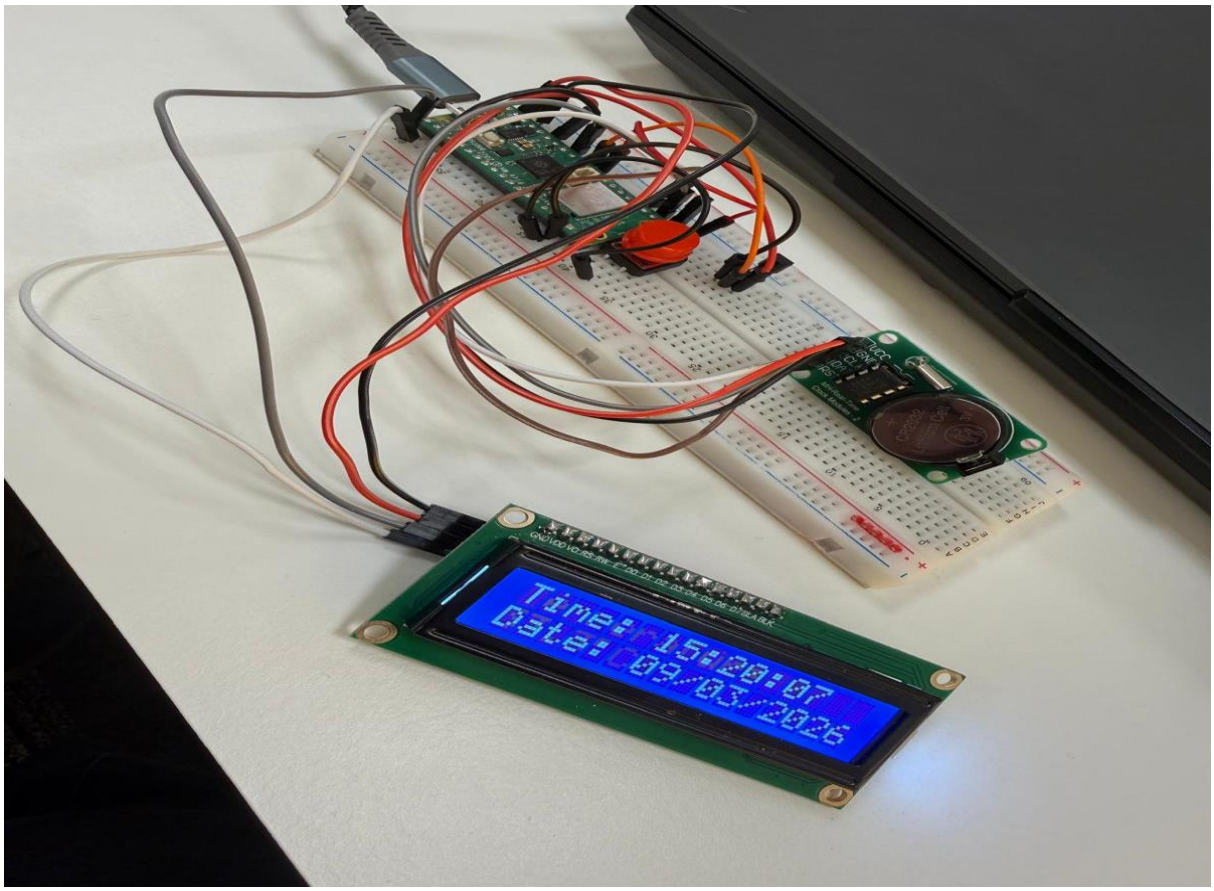
    else:
        # --- TEMPERATURE SCREEN ---
        raw = lm35.read_u16()
        voltage = (raw / 65535) * 3.3
        temp_c = voltage * 100
```

```
        lcd.move_to(0,0)
        lcd.putstr("Temperature:")

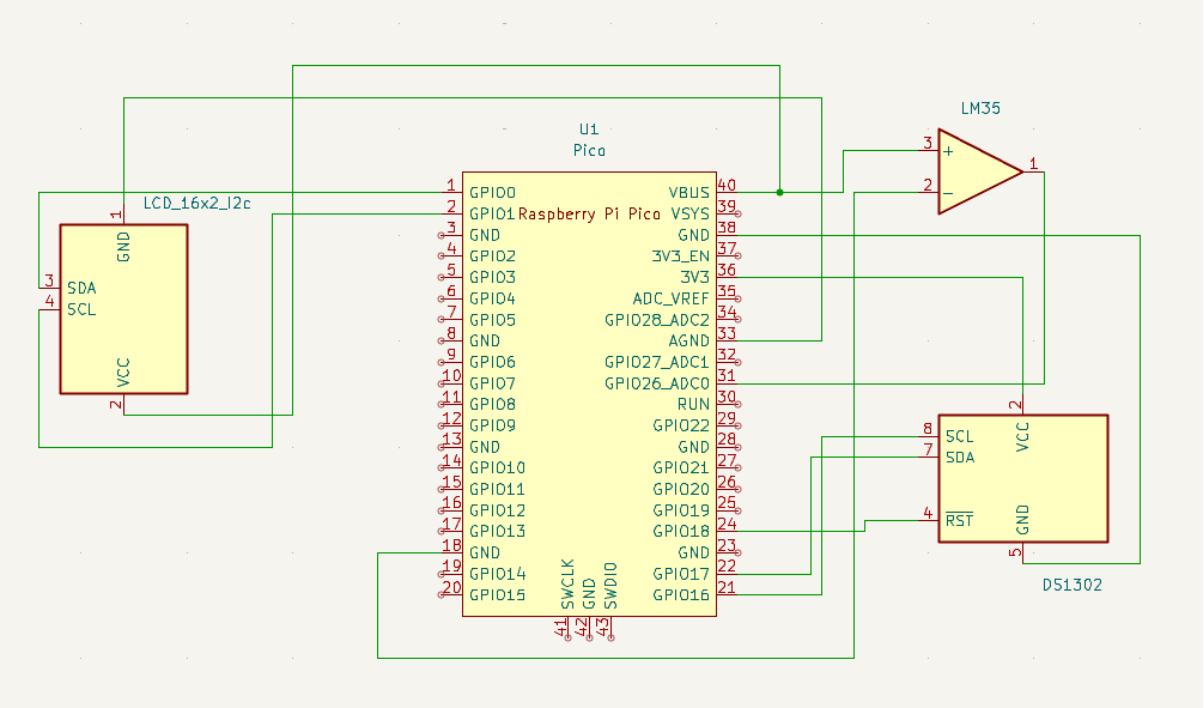
        lcd.move_to(0,1)
        lcd.putstr("{:.1f}".format(temp_c))
        lcd.putstr(chr(223)) # degree symbol
        lcd.putstr("C")

    utime.sleep(0.2)
```

Image of the project:



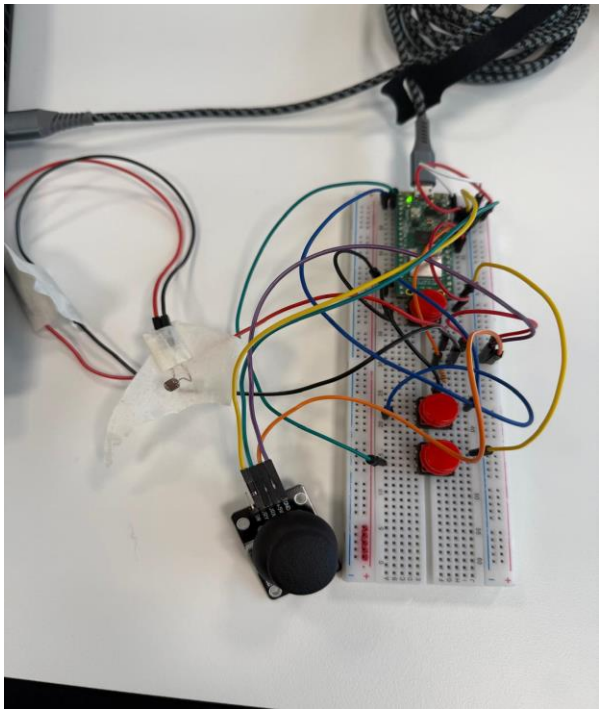
Schematic: using KiCad



Assignments Week 3.4:

1) Dino cheater
2) Analog joystick

Image for both projects:



1) For Dino Cheater the assignment is to play the dino chrome game without using the keyboard:
   I used a light sensor to detect the cactus and send UP Arrow key to jump

The code:

```
import usb_hid
import time
import board
import analogio
import digitalio
from adafruit_hid.keyboard import Keyboard
from adafruit_hid.keyboard import Keycode

# Keyboard HID
kbd = Keyboard(usb_hid.devices)

# LDR to GP26 (A0)
light = analogio.AnalogIn(board.A0)

# starting button
btn = digitalio.DigitalInOut(board.GP15)
btn.switch_to_input(pull=digitalio.Pull.UP)

# changable threshold
THRESHOLD =  41000

def get_light():
    return light.value  # 0 -  65535

print("Press the button to start...")

while btn.value: # True = not pressed
    time.sleep(0.01)

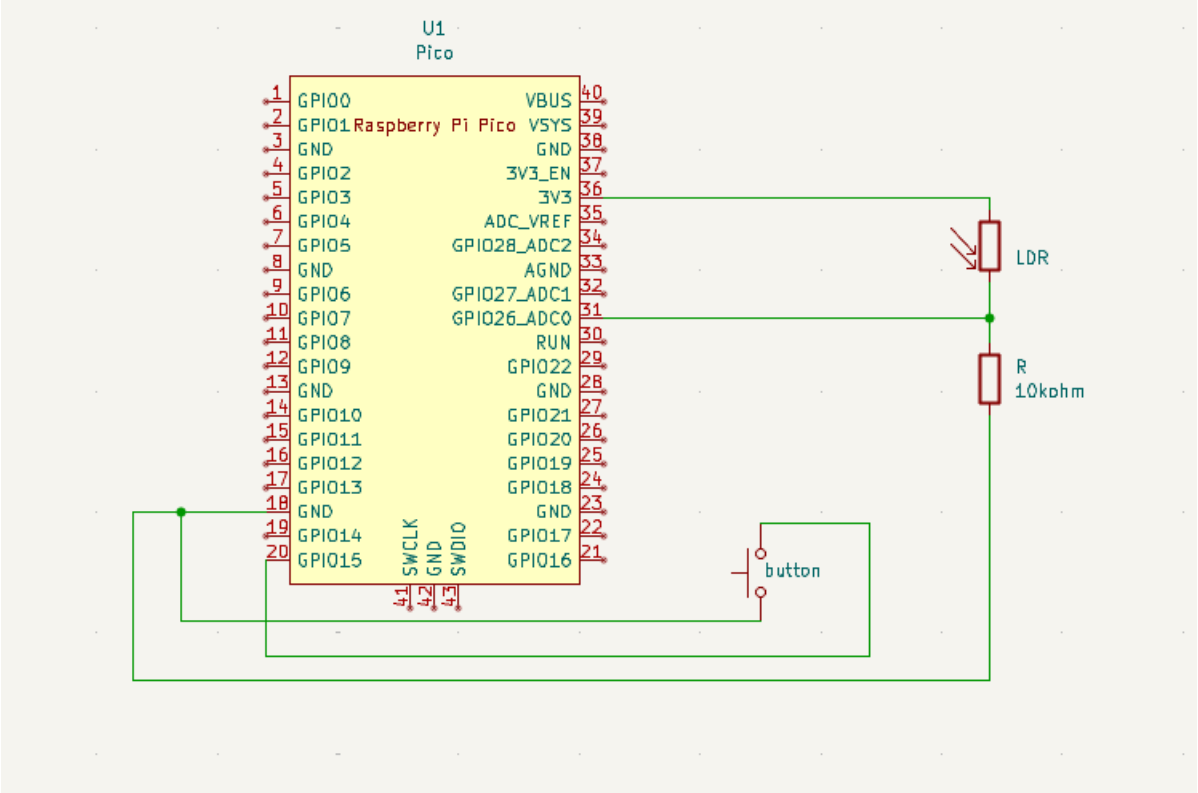
print("Button pressed! Starting in 1 sec...")
time.sleep(1)

while True:

    brightness = get_light()
    print(brightness) # help find threshold

    # when screen get darker than threshold press happen
    if brightness < THRESHOLD:
        kbd.press(Keycode.UP_ARROW)
        kbd.release(Keycode.UP_ARROW)
        time.sleep(0.10) # small delay to avoid double jump
    time.sleep(0.001) # to reduce CPU heat
```

The Schematic:



2) For the analog joystick assignment, we have to play a game using a joystick and buttons.

The code:

```
import usb_hid
import time
import board
import analogio
import digitalio
from adafruit_hid.keyboard import Keyboard
from adafruit_hid.keyboard import Keycode

kbd = Keyboard(usb_hid.devices)

# joystick axes
x_axis = analogio.AnalogIn(board.A1) # ADC1
y_axis = analogio.AnalogIn(board.A2) # ADC2

# jump button
```

```
jump_btn = digitalio.DigitalInOut(board.GP15)
jump_btn.switch_to_input(pull=digitalio.Pull.UP)

#fire button
fire_btn = digitalio.DigitalInOut(board.GP16)
fire_btn.switch_to_input(pull=digitalio.Pull.UP)



# Threshold for movement
LEFT_THRESHOLD = 400
RIGHT_THRESHOLD = 45000
DOWN_THRESHOLD = 45000

while True:

    x = x_axis.value
    y = y_axis.value

    # turn left
    if x < LEFT_THRESHOLD:
        kbd.press(Keycode.LEFT_ARROW)
    else:
        kbd.release(Keycode.LEFT_ARROW)

    # turn right
    if x > RIGHT_THRESHOLD:
        kbd.press(Keycode.RIGHT_ARROW)
    else:
        kbd.release(Keycode.RIGHT_ARROW)

    # down
    if y > DOWN_THRESHOLD:
        kbd.press(Keycode.DOWN_ARROW)
    else:
        kbd.release(Keycode.DOWN_ARROW)

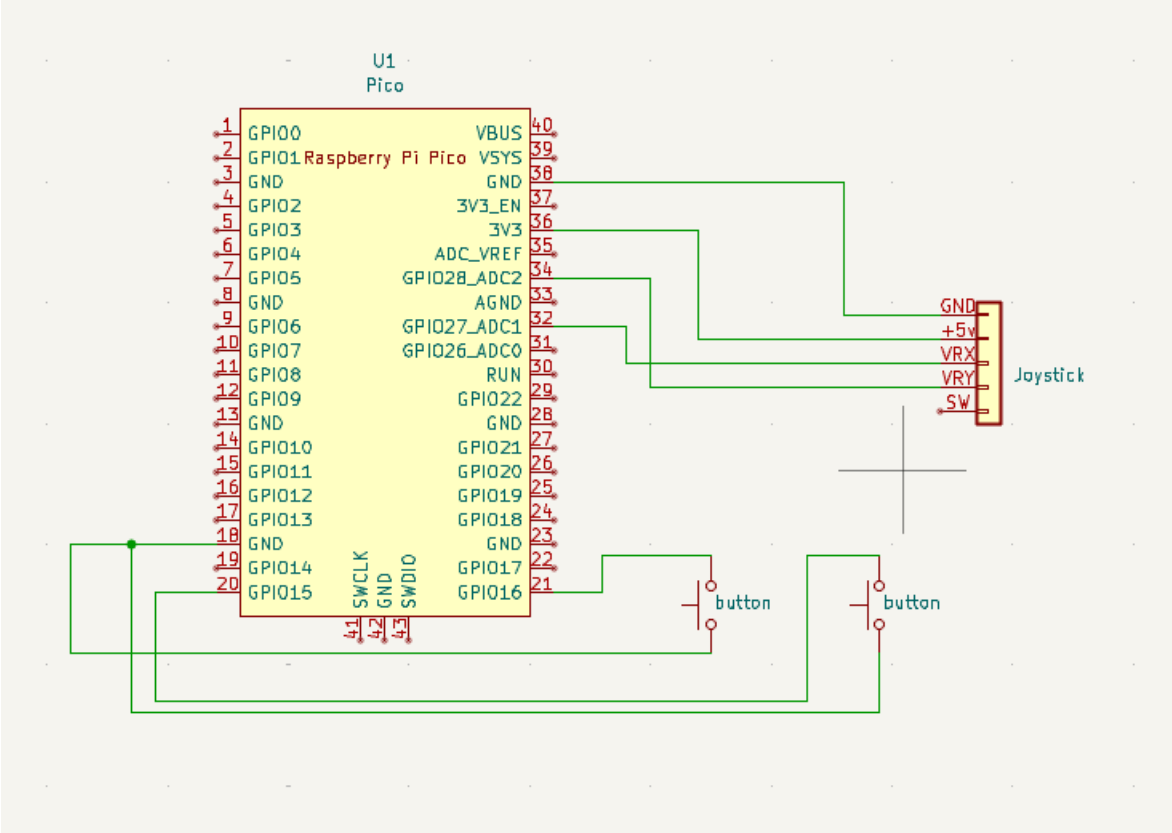
    # jump
    if not jump_btn.value: # when button pressed
        kbd.press(Keycode.SPACE)
    else:
        kbd.release(Keycode.SPACE)

    # fire
    if not fire_btn.value:
        kbd.press(Keycode.SHIFT)
    else:
```

```
        kbd.release(Keycode.SHIFT)

    time.sleep(0.01) # reduce heat
```

The schematic:



Assignments Week 3.5:

1) Wi-Fi scanner

2) Controlling stuff (web server)

1- Wi-Fi scanner:

The assignment is to create a program that show the nearby networks in a table sorted by the RSSI strength:

The code:

```
import network
import utime
```

```
def main():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    while True:
        data = wlan.scan()
        # Sort by RSSI (strongest first)
        data.sort(key=lambda d: d[3], reverse=True)
        scan(data)
        utime.sleep(10)

def scan(data):
    # Table Header
    print("{:<6}| {:<30} |{:<4}".format("Number", "Network", "RSSI"))
    print("-" * 50)

    for i, d in enumerate(data, start=1):
        ssid = d[0].decode() # from bytes to string
        rssi = d[3]
        print("{:<6}| {:<30} |{:<4}".format(i, ssid, rssi))

    print()
main()
```

Image of the terminal:



2 – Controlling stuff (webserver)

The assignment is to create a webserver that can control something and in my case it was 2 LEDs (green – red) and physical component that control something in the webserver in my case it was a physical button that control the background of the website between dark and light modes

The code:

```
from machine import Pin
import network
import socket
import utime
```

```
# configure the LED
LED_RED = Pin(14, Pin.OUT)
LED_GREEN = Pin(15, Pin.OUT)

# configure the btn
BTN = Pin(17, Pin.IN, Pin.PULL_UP)

# Dark mode state
dark_mode = False



# setup for the LED all off at the start
LED_RED.value(0)
LED_GREEN.value(0)

def main():
    global dark_mode
    connect()
    addr = socket.getaddrinfo("0.0.0.0",80)[0][-1]
    # create the server
    s = socket.socket()
    s.bind(addr)
    # server listen for one client
    s.listen(1)

    while True:
        if BTN.value() == 0: # pressed
            dark_mode = not dark_mode # toggle
            utime.sleep(0.3)

        conn, address = s.accept()
        request = conn.recv(1024)
        request = str(request)

        LED_RED_ON = request.find("/?LED_R=ON")
        LED_RED_OFF = request.find("/?LED_R=OFF")
        LED_GREEN_ON = request.find("/?LED_G=ON")
        LED_GREEN_OFF = request.find("/?LED_G=OFF")

        if LED_RED_ON != -1:
            LED_RED.value(1)
        if LED_RED_OFF != -1:
            LED_RED.value(0)
        if LED_GREEN_ON != -1:
            LED_GREEN.value(1)
```

```
        if LED_GREEN_OFF != -1:
            LED_GREEN.value(0)

        # for LED commands (no page reload)
        if (LED_RED_ON != -1 or LED_RED_OFF != -1 or
            LED_GREEN_ON != -1 or LED_GREEN_OFF != -1):
            conn.send("OK")
            conn.close()
            continue

        if "/mode" in request:
            if dark_mode:
                conn.send("dark")
            else:
                conn.send("light")
            conn.close()
            continue

        conn.send(html)
        conn.close()

def connect():
    # create station network interface
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    # connect to my hotspot
    wlan.connect("Slman","00000000")
    while wlan.isconnected() == False:
        print("Waiting to be connected")
        utime.sleep(0.5)



    print("Connected")
    print(wlan.ifconfig())

html = """<!DOCTYPE html>
<html>
<head>
<style>
body {
    transition: background-color 0.3s, color 0.3s;
}
</style>

<script>
/*
```

```
    updateMode() asks the Pico whether dark mode is active.
    The Pico returns "dark" or "light", and we update the page colors.
*/
function updateMode() {
    fetch("/mode")
        .then(r => r.text()) // Take the response object and extract the
actual text inside it
        .then(mode => {
            if (mode === "dark") {
                document.body.style.backgroundColor = "black";
                document.body.style.color = "white";
            } else {
                document.body.style.backgroundColor = "white";
                document.body.style.color = "black";
            }
        });
}

function led(which, state) {
    fetch("/?" + which + "=" + state);
}

setInterval(updateMode, 300);
</script>
</head>

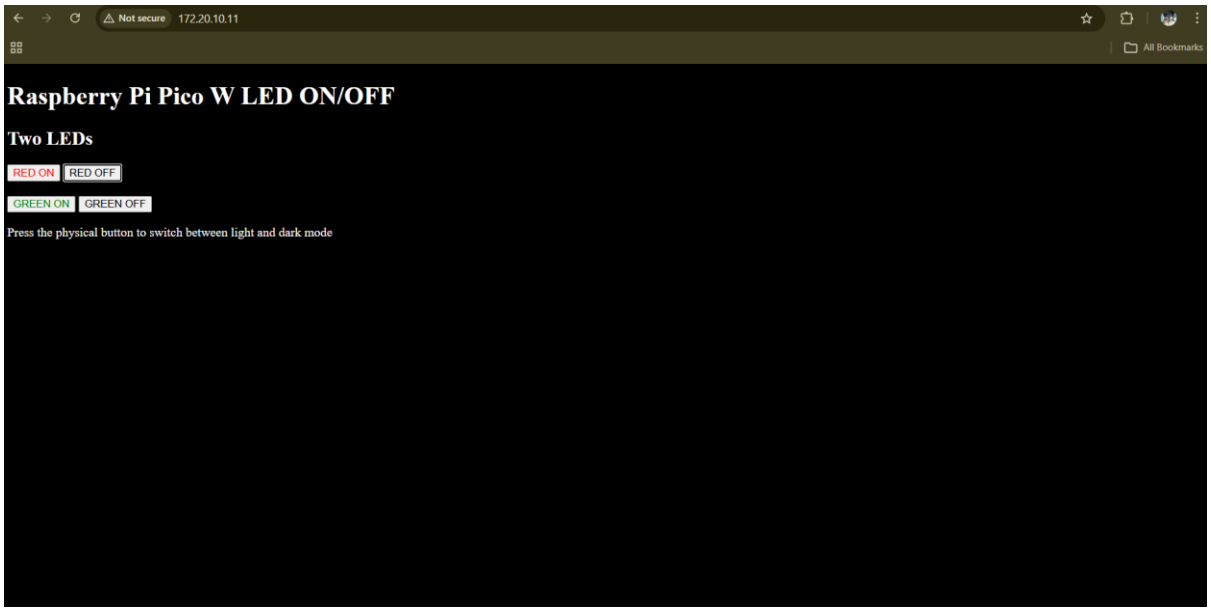
<body>
<h1>Raspberry Pi Pico W LED ON/OFF</h1>
<h2>Two LEDs</h2>

<button onclick="led('LED_R','ON')" style="color:red">RED ON</button>
<button onclick="led('LED_R','OFF')">RED OFF</button><br><br>

<button onclick="led('LED_G','ON')" style="color:green">GREEN ON</button>
<button onclick="led('LED_G','OFF')">GREEN OFF</button>

<p>Press the physical button to switch between light and dark mode</p>
</body>
</html>
"""
main()
```

Image of the website:



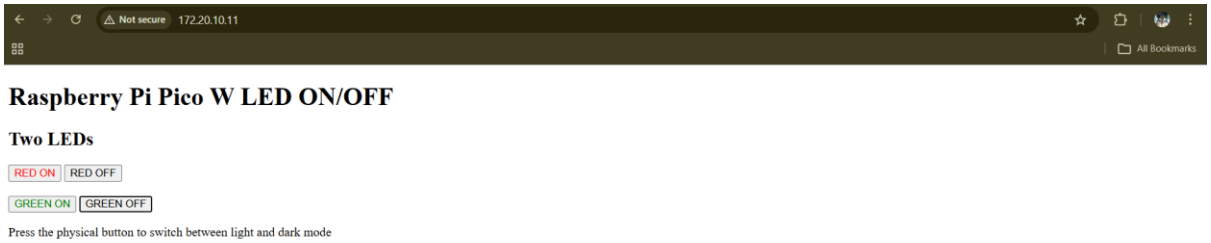
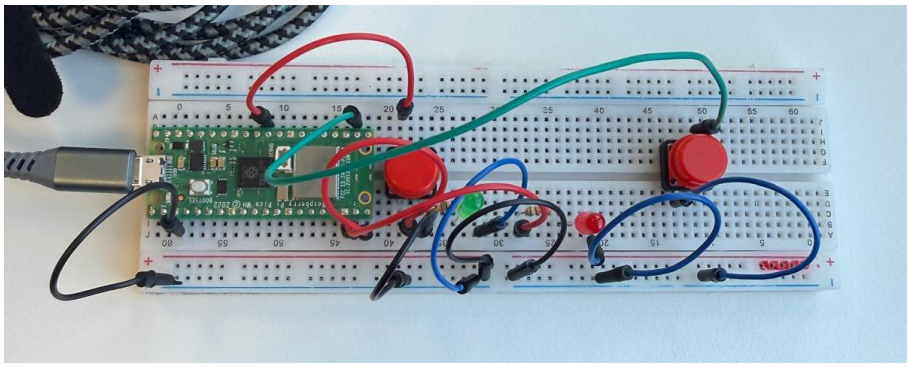
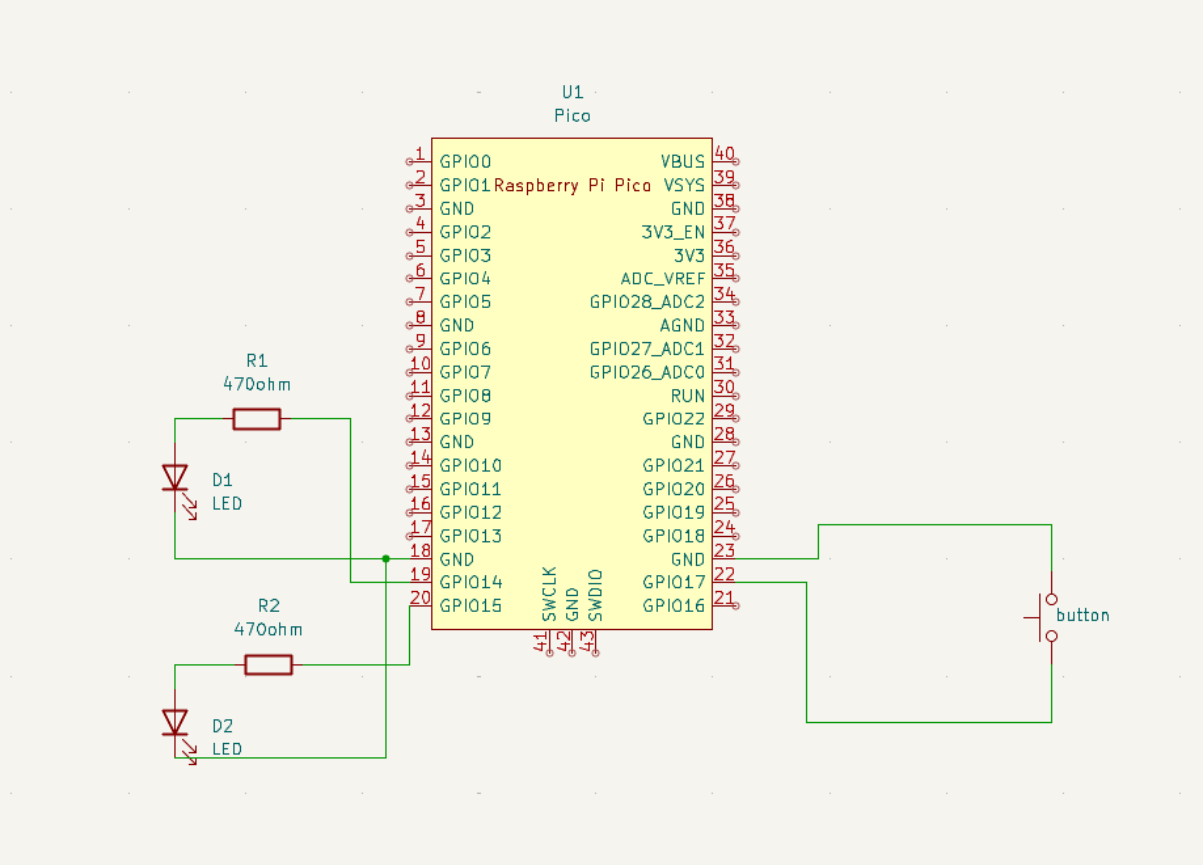


Image of the wiring:



Kicad Schematic:



Assignments Week 3.6:

1) Attendance:

   In this assignment, I have to design and build a complete RFID-based attendance system using a Raspberry Pico W, an MFRC522 RFID reader, an LCD display, an RTC module, LEDs, a buzzer, and a simple web interface. The goal is to scan RFID cards, register new users, and track their clock-in and clock-out actions with accurate timestamps. I programmed the Pico W to run in two modes: a detect mode for normal scanning and a register mode for adding new users. Each action is logged with the user's name, the action type, and the time, and this information is shown on a webpage hosted directly by the Pico W. The webpage automatically refreshes every few seconds and displays the log in a color-coded table, where green indicates a clock-in, red indicates a clock-out, and yellow shows newly registered users. The project combines hardware interaction, real-time feedback through LEDs and sound, and a lightweight web server, giving me hands-on experience with embedded systems, networking, and Python programming.

   The code:

```
from mfrc522 import MFRC522
```

```
from machine import Pin, I2C, PWM
from lcd_api import LcdApi
from i2c_lcd import I2cLcd
import ds1302
import network
import socket
import time

# ----------------------------------------
# LCD SETUP
# ----------------------------------------
# The LCD uses I2C, so we set up the I2C pins and address.
I2C_ADDR = 0x27
i2c = I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
lcd = I2cLcd(i2c, I2C_ADDR, 2, 16)  # 2 rows, 16 columns

# ----------------------------------------
# REAL TIME CLOCK (RTC)
# ----------------------------------------
# This gives us the current date/time for logging.
rtc = ds1302.DS1302(Pin(2), Pin(3), Pin(4))

def get_timestamp():
    # Format the time nicely for the table.
    return "%04d-%02d-%02d %02d:%02d:%02d" % (
        rtc.year(), rtc.month(), rtc.day(),
        rtc.hour(), rtc.minute(), rtc.second()
    )

# ----------------------------------------
# LED SETUP (RGB LED)
# ----------------------------------------
# We use PWM so we can control brightness.
led_r = PWM(Pin(12))
led_g = PWM(Pin(11))
led_b = PWM(Pin(10))

# Set PWM frequency for all LEDs.
for l in (led_r, led_g, led_b):
    l.freq(1000)

def led_off():
    # Turn off all LED colors.
    led_r.duty_u16(0)
    led_g.duty_u16(0)
    led_b.duty_u16(0)
```

```
def led_red():
    led_off()
    led_r.duty_u16(65535)

def led_green():
    led_off()
    led_g.duty_u16(65535)

def led_yellow():
    # Mix red + green to get yellow.
    led_off()
    led_r.duty_u16(50000)
    led_g.duty_u16(10000)

# ----------------------------------------
# BUZZER
# ----------------------------------------
# Simple PWM buzzer for sound feedback.
buzzer = PWM(Pin(14))
buzzer.freq(1000)

def buzz(freq, ms):
    buzzer.freq(freq)
    buzzer.duty_u16(30000)
    time.sleep_ms(ms)
    buzzer.duty_u16(0)

# Different sound effects for actions.
def tone_scan(): buzz(2000, 60)
def tone_granted(): buzz(1500, 80); buzz(2000, 80)
def tone_denied(): buzz(400, 300)
def tone_clockout(): buzz(800, 120); buzz(500, 150)

# ----------------------------------------
# RFID READER SETUP
# ----------------------------------------
reader = MFRC522(
    spi_id=0,
    sck=18, miso=16, mosi=19,
    cs=17, rst=20
)
reader.antenna_on()

# ----------------------------------------
# SYSTEM STATE
```

```
# ----------------------------------------
button = Pin(15, Pin.IN, Pin.PULL_UP)  # Button to switch modes

mode = "detect"         # detect or register mode
current_user = None     # name typed during registration
users = {}              # username -> list of card IDs
attendance = {}         # username -> IN/OUT
log = []                # list of (name, action, timestamp)

# ----------------------------------------
# LCD HELPER FUNCTIONS
# ----------------------------------------
def show_detect_mode():
    lcd.clear()
    lcd.putstr("Mode: DETECT")
    led_yellow()

def show_reg_mode():
    lcd.clear()
    lcd.putstr("REG MODE")
    led_yellow()

def show_message(a, b="", color="yellow"):
    # Show two-line message on LCD.
    lcd.clear()
    lcd.putstr(a[:16])
    lcd.move_to(0, 1)
    lcd.putstr(b[:16])

    # LED color feedback.
    if color == "green":
        led_green()
    elif color == "red":
        led_red()
    else:
        led_yellow()

# ----------------------------------------
# REGISTER A NEW CARD
# ----------------------------------------
def register_card(cid):
    global current_user

    # Check if card already belongs to someone.
    for user, cards in users.items():
        if cid in cards:
```

```
            show_message("Card belongs to:", user, "red")
            tone_denied()
            time.sleep(1)
            show_reg_mode()
            return

    # Add new user if not in dictionary.
    if current_user not in users:
        users[current_user] = []

    users[current_user].append(cid)

    # Add registration to the table.
    log.append((current_user, "REGISTERED", get_timestamp()))

    show_message("Added to:", current_user, "green")
    tone_granted()
    time.sleep(1)
    show_reg_mode()

# ----------------------------------------
# CLOCK IN / CLOCK OUT
# ----------------------------------------
def process_detection(cid):
    for u, cards in users.items():
        if cid in cards:
            ts = get_timestamp()

            # Toggle IN/OUT state.
            if u not in attendance or attendance[u] == "OUT":
                attendance[u] = "IN"
                action = "CLOCK IN"
                tone_granted()
            else:
                attendance[u] = "OUT"
                action = "CLOCK OUT"
                tone_clockout()

            # Add to log table.
            log.append((u, action, ts))

            show_message(u, action, "green")
            time.sleep(1)
            show_detect_mode()
            return
```

```
    # If card not found.
    show_message("Access Denied", "", "red")
    tone_denied()
    time.sleep(1)
    show_detect_mode()

# ----------------------------------------
# WIFI CONNECTION
# ----------------------------------------
def connect_wifi():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)
    wlan.connect("Slman", "00000000")  # your WiFi
    while not wlan.isconnected():
        print("Connecting...")
        time.sleep(0.5)
    print("Connected:", wlan.ifconfig())

# ----------------------------------------
# BUILD TABLE ROWS (HTML)
# ----------------------------------------
def build_table_rows():
    rows = "<tr><th>Name</th><th>Action</th><th>Timestamp</th></tr>"

    # Loop newest → oldest
    for u, a, ts in reversed(log):

        # Color coding for table rows.
        if a == "CLOCK IN":
            style = "style='background:#003300; color:#00ff00;'"
        elif a == "CLOCK OUT":
            style = "style='background:#330000; color:#ff4444;'"
        elif a == "REGISTERED":
            style = "style='background:#333300; color:#ffff66;'"
        else:
            style = ""

        rows += "<tr %s><td>%s</td><td>%s</td><td>%s</td></tr>" % (
            style, u, a, ts
        )

    return rows

# ----------------------------------------
# FULL HTML PAGE (AUTO REFRESH)
# ----------------------------------------
```

```
def build_page():
    # The page reloads every 3 seconds.
    return """<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Attendance Log</title>
<meta http-equiv="refresh" content="3">
<style>
body { font-family: Arial; background:#111; color:#eee; }
table { width:100%%; border-collapse:collapse; margin-top:20px; }
th,td { border:1px solid #444; padding:8px; }
th { background:#333; }
tr:nth-child(even) { background:#222; }
</style>
</head>
<body>

<h1>Attendance Log</h1>

<table>
%s
</table>

</body>
</html>
""" % build_table_rows()

# ----------------------------------------
# MAIN PROGRAM LOOP
# ----------------------------------------
def main():
    global mode, current_user

    connect_wifi()

    # Start web server.
    addr = socket.getaddrinfo("0.0.0.0", 80)[0][-1]
    s = socket.socket()
    s.bind(addr)
    s.listen(1)
    s.settimeout(0)  # non-blocking

    print("Server running on:", addr)

    show_detect_mode()
```

```
last_button = 1

while True:

    # -------------------------
    # BUTTON: SWITCH MODES
    # -------------------------
    if last_button == 1 and button.value() == 0:
        if mode == "detect":
            mode = "register"
            show_reg_mode()
            current_user = input("Enter username: ")
            show_message("User:", current_user)
        else:
            mode = "detect"
            show_detect_mode()
        time.sleep(0.25)
    last_button = button.value()

    # -------------------------
    # RFID SCAN
    # -------------------------
    stat, _ = reader.request(reader.REQIDL)
    if stat == reader.OK:
        tone_scan()
        led_yellow()
        stat, uid = reader.SelectTagSN()
        if stat == reader.OK:
            cid = int.from_bytes(bytes(uid), "little", False)
            if mode == "register":
                register_card(cid)
            else:
                process_detection(cid)
        time.sleep(0.5)

    # -------------------------
    # WEB SERVER RESPONSE
    # -------------------------
    try:
        conn, caddr = s.accept()
    except:
        conn = None

    if conn:
        try:
```

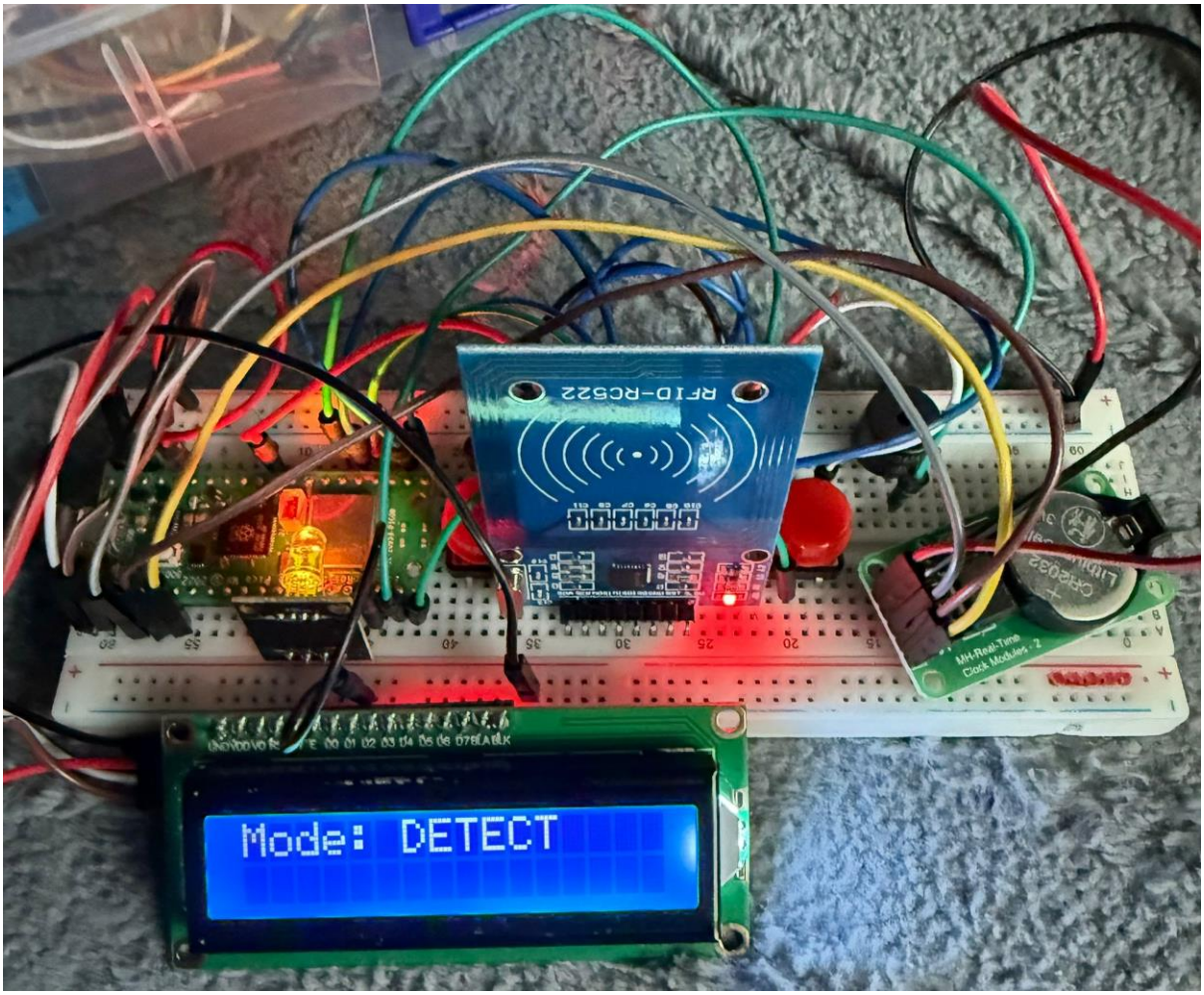
```
            request = conn.recv(1024).decode()
        except:
            request = ""

        # Always send full page.
        conn.send("HTTP/1.1 200 OK\r\n")
        conn.send("Content-Type: text/html\r\n")
        conn.send("Connection: close\r\n\r\n")
        conn.send(build_page())
        conn.close()

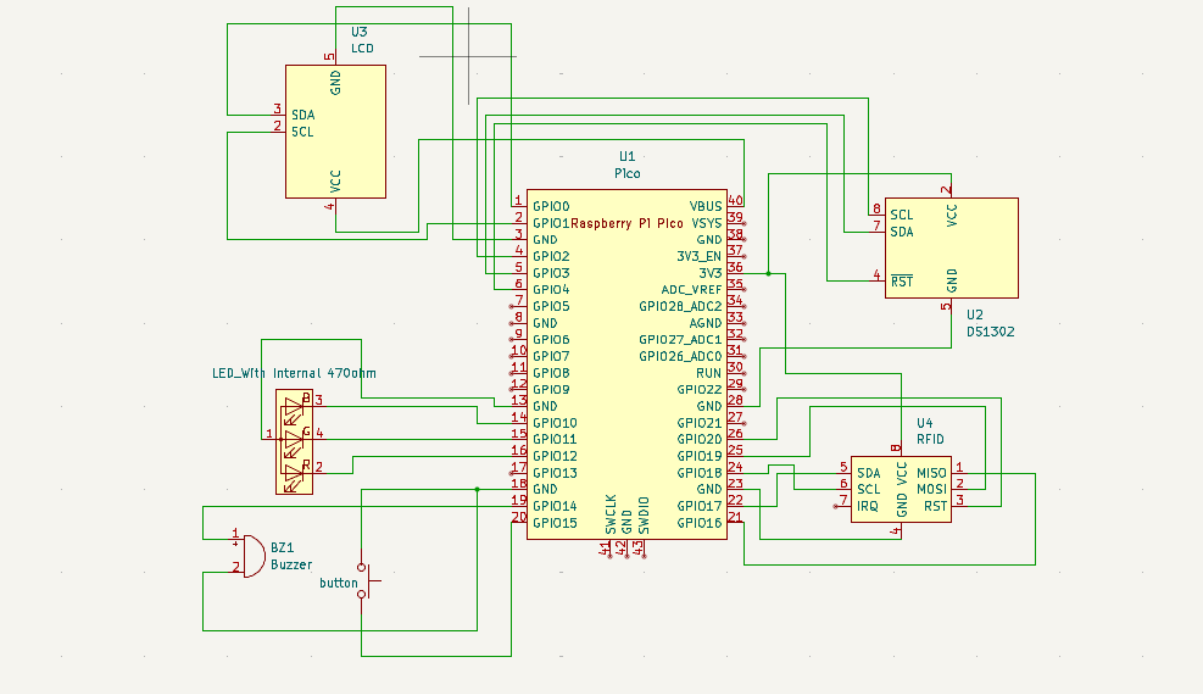
main()
```

Pictures of the project:





Kicad Schematic:



NHL STENDEN university of applied sciences

P3. Embedded Systems

Name	Student nr.	Group	Year
Slman Hamoad	5638364		

The student is responsible for safe keeping of this card

Assignments

Wk	Ch	Subject	Rating	Date	Signature
3.1	1/3	Blink with external hardware reset	(V) O	9 FEB 2026	M. Baro
	2/3	Mario Pico	(V) O	9 FEB 2026	M. Baro
3.2	4/5	7-Segments Voltmeter *(0…10V)*	(G) V O	19 FEB 2026	M. Baro
3.3	5	RTC and temperature on LCD-display	G (V) O	9 MAR 2026	M. Baro
3.4	-	Dino cheater *(HID)*	G (V) O	11 MAR 2026	M. Baro
		Analog joystick *(HID)*	G (V) O	11 MAR 2026	M. Baro
3.5	11	Wifi scanner	G (V) O	19 MAR 2026	M. Baro
		Controlling stuff *(Webserver)*	G (V) O	19 MAR 2026	M. Baro
3.6 3.7	12	Attendance *(RFID, RTC, web and LCD)*	G (V) O	30 MAR 2026	M. Baro
3.8	-	Solo Pico Project *(not mandatory)*	U G V O		
3.8	-	Completing Portfolio			
T3	-	**Assessment**			

U = Ultimate, G = Good, V = Valid, O = Off standard

Feedback:

3.2 + OL

Voldaan
30 MAR 2026
M. Baro

This card is proof that the above assignments are completed and should be:

- signed off during guided ateliers by the assigned teacher(s), in the same week,
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment

P3. Computer Science

Name	Student nr.	Group	Year
Slman Hamood	5638364		

The student is responsible for safe keeping of this card

Assignments

Wk	Subject	Date	Signature
3.1	Smart Café Helper	19 FEB 2026	
	Vanity Plates goes Flask + venv + DB	19 FEB 2026	
3.2	Manic Monday Functions and Variables	19 FEB 2026	
	Survival Simulator + real survivor	19 FEB 2026	
3.3	Manic Monday	19 FEB 2026	
	Testing my twttr unittest battle VERY Good	19 FEB 2026	
	Back to the Bank	9 MAR 2026	
	Re-requesting a vanity plate	9 MAR 2026	
	Nutrition Facts	9 MAR 2026	
3.4	Presentation	9 MAR 2026	
3.5	Scavenger hunt → 30 mostly 46		
	CS50 P-Shirt	16 MAR 2026	
3.6	Manic Monday	16 MAR 2026	
	~~Scavenger hunt Pt II~~		
	Product goes OOP		
	Tic-Tac-Toe		
3.7	The Misleading Minesweeper		
3.8	Manic Monday RESIT		
	Presentation Solo Innovator Project		
	Check-Check-Triple **Check**		
T3	**Assessment**		

This card is proof that the above assignments are properly completed and should be:

- signed off by the appropriate teacher(s) during the guided ateliers, the same week
- a digital attachment of your portfolio and
- physically handed over at the beginning of the assessment